

CHAPTER 2

COMPUTER ORGANIZATION AND ARCHITECTURE

Syllabus: Machine instructions and addressing modes, ALU and data path, CPU control design, memory interface, I/O interface (interrupt and DMA mode), instruction pipelining, cache and main memory, secondary storage

2.1 INTRODUCTION

Computer architecture and organization is the science of interconnecting hardware components, designing and configuring the hardware/software interface to fulfill functional and performance goals of a computer. This chapter outlines the basic hardware structure of a modern digital programmable computer, the basic laws for performance evaluation, designing the control and data path hardware for a processor, concept of pipelining for executing machine instructions simultaneously and designing fast memory and storage systems.

Computer architecture deals with the structure and behaviour of computer system as viewed by the user. It encompasses instruction formats, the instruction set architecture (ISA) and addressing modes.

Computer organization deals with the operation and interconnection of the various hardware components.

2.2 COMPUTER ARCHITECTURE

2.2.1 Register Set

The computer needs registers for processing and manipulating data and for holding memory addresses that are available to the machine-code programmer. Some registers for a basic computer are given in Table 2.1.

Table 2.1 | Types of registers and their functions

Register Symbol	Register Name	Function
DR	Data register	Holds memory operand
ACC	Accumulator	Special purpose processor register
AR	Address register	Holds address for memory

(Continued)

Table 2.1 | Continued

Register Symbol	Register Name	Function
IR	Instruction register	Holds an instruction that is to be executed
PC	Program counter	Holds address of instruction to be executed next
TR	Temporary register	Holds temporary data if required
INPR	Input register	Holds input character
OUTR	Output register	Holds output character

2.2.2 Quantitative Principles to Design High-Performance Processor

Amdahl's law focused on performance gain after enhancing the system. The performance gain is denoted by S_{overall} and ET stands for execution time.

$$S_{\text{overall}} = \frac{\text{Performance of the system with enhancement}}{\text{Performance of the system without enhancement}}$$

$$S_{\text{overall}} = \frac{1/\text{ET}_{\text{new}}}{1/\text{ET}_{\text{old}}} \quad (2.1)$$

$$S_{\text{overall}} = \frac{\text{ET}_{\text{old}}}{\text{ET}_{\text{new}}}$$

After enhancement, the system consists of two portions: unenhanced and enhanced portion.

$$\text{ET}_{\text{new}} = \text{ET of the unenhanced portion} + \text{ET of enhanced portion}$$

To calculate ET_{new} , the following two factors are needed:

1. **Fraction_{enhance} (F):** It indicates how much portion of the old system undergoes enhancement.
2. **Speed_{enhance} (S):** It indicates how many times the new portion is running faster than the old portion.

$$S = \frac{\text{Performance}_{\text{new}} F}{\text{Performance}_{\text{old}} F} = \frac{1/\text{ET}_{\text{new}} F}{1/\text{ET}_{\text{old}} F} = \frac{\text{ET}_{\text{old}} F}{\text{ET}_{\text{new}} F}$$

$$\text{So, } \text{ET}_{\text{new}} F = \frac{\text{ET}_{\text{old}} F}{S}$$

On the basis of the above factor,

$$\text{ET}_{\text{new}} F = \text{ET}_{\text{old}}(1 - F) + \frac{\text{ET}_{\text{old}} F}{S}$$

Substitute the value of ET in Eq. (2.1):

$$\text{ET}_{\text{new}} F = \text{ET}_{\text{old}}(1 - F) + \frac{\text{ET}_{\text{old}} F}{S}$$

Let $\text{ET}_{\text{old}} = 1$,

$$S_{\text{overall}} = \frac{1}{(1 - F) + (F/S)} = \left[(1 - F) + \frac{F}{S} \right]^{-1}$$

When the system consists of multiple frequent cases, where i is the number of frequent cases:

$$S_{\text{overall}} = \left[(1 - \sum F_i) + \sum \frac{F_i}{S} \right]^{-1}$$

Problem 2.1: Consider a hypothetical processor used in mathematical model simulation. It consists of two functional units, floating point and integer. The floating point is enhanced then it runs two times faster, but only 10% of the instructions are floating point. What is the speed up?

Solution: Here $S = 2$, $F = 0.1$

$$S_{\text{overall}} = \left[(1 - 0.1) + \frac{0.1}{2} \right]^{-1} = 1.052$$

2.3 MACHINE INSTRUCTIONS AND ADDRESSING MODES

Machine instruction is an individual machine code. The complete set of all machine codes recognized by a particular processor makes its Instruction Set. Instructions can be grouped according to the function they perform. The number of ways by which arguments for these machine instructions can be specified constitutes the addressing modes for a processor.

2.3.1 Machine Instructions

An instruction is a command to the microprocessor to perform a given task. Most computer instructions are classified as follows:

1. **Data transfer instructions:** These instructions move data from one place to another in the computer without changing the data content. Example: LOAD, MOVE, IN, OUT, PUSH, STORE.
2. **Data manipulation instructions:** These instructions perform arithmetic, logical and shift operations on data. Example: ADD, SUB, MUL, DIV, INC, AND, XOR, OR, SHR, SHL, ROR, ROL.
3. **Program control instructions:** These instructions may change the address value in program counter and cause the normal sequential flow to change.

On the basis of the number of address fields in an instruction, they are classified as follows:

1. **Three-address instruction:** Computer with three-address instruction format can use each address field to specify two sources and a destination, which can be either a processor register or a memory operand. It results in short program but requires too many bits to specify three addresses. Example: ADD R_1, A, B ($R_1 \leftarrow M[A] + M[B]$)

- 2. Two-address instruction:** Each address field can specify either a processor register or a memory word.

Example: $\text{MOV } R_1, A \quad (R_1 \leftarrow M[A]);$
 $\text{MUL } R_1, R_2 \quad (R_1 \leftarrow R_1 * R_2)$

- 3. One-address instruction:** It used an implied accumulator (AC) register for all data manipulation. The other operand is in register or memory.

Example: $\text{LOAD } A \quad (\text{AC} \leftarrow M[A]);$
 $\text{ADD } B \quad (\text{AC} \leftarrow \text{AC} + M[B])$

- 4. Zero-address instruction:** A stack organized computer does not use an address field for the instruction ADD and MUL.

Problem 2.2:

- (a) ISA of a processor consists of 64 registers, 125 instructions and 8 bits for immediate mode. In a given program, 30% of the instructions take one input register and have one output register, 30% have two input registers and one output register, 20% have one immediate input, and one output register, and remaining have two immediate input, 1 register input and one output register. Calculate the number of bits required for each instruction type. Assume that the ISA requires that all instructions be a multiple of 8 bits in length.
- (b) Compare the memory space required with that of variable length instruction set.

Solution:

- (a) Since there are 125 instructions so we need 7 bits to differentiate them as $64 < 125 < 128$. For 64 registers, we need 6 bits and 8 bits for immediate mode.
- For Type 1, 1 reg in, 1 reg out: $7 + 6 + 6 = 19 \text{ bits} \sim 32 \text{ bits}$
 For Type 2, 2 reg in, 1 reg out: $7 + 6 + 6 + 6 = 26 \text{ bits} \sim 32 \text{ bits}$
 For Type 3, 1 imm in, 1 reg out: $7 + 6 + 8 = 21 \text{ bits} \sim 32 \text{ bits}$
 For Type 4, reg in, 2 imm in, 1 reg out: $7 + 6 + 8 + 8 + 6 = 35 \text{ bits} \sim 48 \text{ bits}$
- (b) As the largest instruction type requires 48 bit instructions, the fixed-length instruction format uses 48 bits per instruction. Variable length instruction format uses $0.3 \times 32 + 0.3 \times 32 + 0.2 \times 32 + 0.2 \times 48 = 36 = \text{bits on average, that is, } 25\% \text{ less space.}$

1. provide programming flexibility to users through use of pointers to memory, counter for loop control, data indexing and program relocation.
2. reduce the size of the addressing field of the instruction.

Let us suppose $[x]$ means contents at location x for all the addressing modes.

2.3.2.1 Types of Addressing Modes

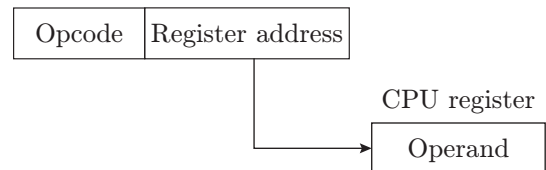
- 1. Implied mode:** In this mode, the operands are implicitly stated in the instruction. For example, register reference instructions such as CMA (complement accumulator), CLA (clear accumulator) and zero-address instructions that use stack organization.
- 2. Immediate mode:** In this mode, the operand is specified in the instruction itself, that is, address field is replaced by an actual operand. Immediate mode instructions are useful for initializing registers to a constant value. For example, used for initializing CPU registers to some constant value such as $\text{MOV } R_1, \#34$.

Instruction with immediate mode



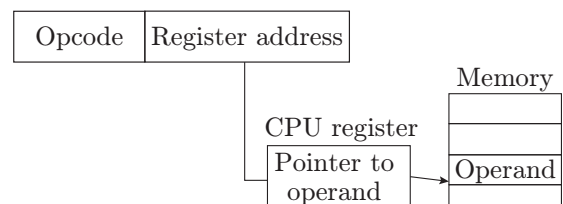
- 3. Register (direct) mode:** In this mode, the operands are in CPU registers. An n -bit register field can specify any one of 2^n registers. Example: $\text{ADD } R_1$ will add the contents of an accumulator and contents of R_1 , that is, $\text{ACC} = [\text{ACC}] + [R_1]$.

Instruction with register direct mode



- 4. Register (indirect) mode:** In this mode, the instruction format specifies a CPU register which contains an effective address of the operand residing in memory. This mode ensures less number of bits to specify a register value than to specify a memory location. Example: $\text{ADD } @R_1$ will add the contents of an accumulator with contents of the register R_1 , that is, $\text{AC} = [\text{ACC}] + [[R_1]]$.

Instruction with register indirect mode



2.3.2 Addressing Modes

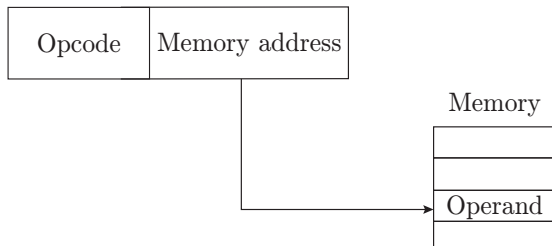
The addressing mode specifies how effective address of an operand is calculated from an instruction. Computers use various addressing mode techniques to:

5. Auto-increment or Auto-decrement mode:

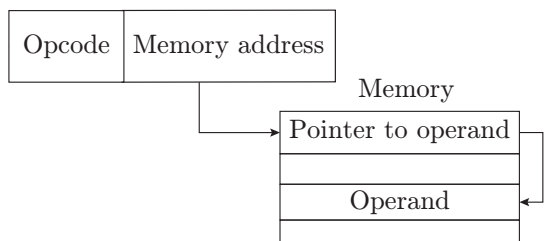
This is similar to register indirect mode except the register containing effective address is incremented or decremented after (or before) its value is used to access memory.

6. Direct address mode: In this mode, the effective address of an operand is equal to the address part of the instruction. Example: ADD *A* instruction adds content of memory cell *A* to accumulator, that is, $ACC = [ACC] + M[A]$.

Instruction with direct address mode

**7. Indirect address mode:** In this mode, memory address specified by address field contains the address of (pointer to) the operand. Example: ADD @ *A* will add the contents of the memory cell *A*, that is, $ACC = [ACC] + M[M[A]]$.

Instruction with indirect address mode

**8. Relative address mode:** In this mode, the effective address of an operand is obtained by adding the content of a program counter to the address part of the instruction. The address part of the instruction can be either positive or negative represented in 2's complement. The result obtained after adding the content of the program counter to the address field produces an effective address whose position in memory is relative to the address of the next instruction.**9. Index address mode:** In this mode, the effective address of an operand is obtained by adding the content of an index register to the address part of the instruction. The index register is a special CPU register that stores an index value and the address field of the instruction stores the base address of a data array in the memory. The distance between the base address and the address of the operand is the index value that is stored in the index register. The index register can be incremented to facilitate access to consecutive operands stored in arrays using the same instruction.**10. Base register addressing mode:** In this mode, the effective address of an operand is obtained by adding the content of a base register to the address part of the instruction. This is somewhat similar to the indexed addressing mode except that the base register stores base or beginning address instead of an index register. It is used for program relocation.

Problem 2.3: A two-word instruction LOAD is stored at location 300 with its address field in the next location. The address field has value 600 and value stored at 600 is 500 and at 500 is 650. The words stored at 900, 901 and 902 are 400, 401 and 402, respectively. A processor register *R* contains the number 800 and index register has value 100. Evaluate the effective address and operand if addressing mode of the instruction is as follows:

1. Direct
2. Indirect
3. Relative
4. Immediate
5. Register indirect
6. Index

Solution: Memory layout is as follows

300	LOAD
301	600
500	650
600	500
700	900
800	700
900	400
901	401
902	402

Addressing Mode	Effective Address	Operand
Direct	600	500
Indirect	500	650
Relative	902	402
Immediate	301	600
Register indirect	800	700
Index	700	900

Problem 2.4: A relative mode branch type instruction is stored in memory at an address equivalent to decimal 600 and the branch is made to an address equivalent to decimal 400. What is the value of the relative address field of the instruction (in decimal)?

Solution: Relative address = $400 - 601 = -201$

Table 2.2 | Arithmetic circuit function table

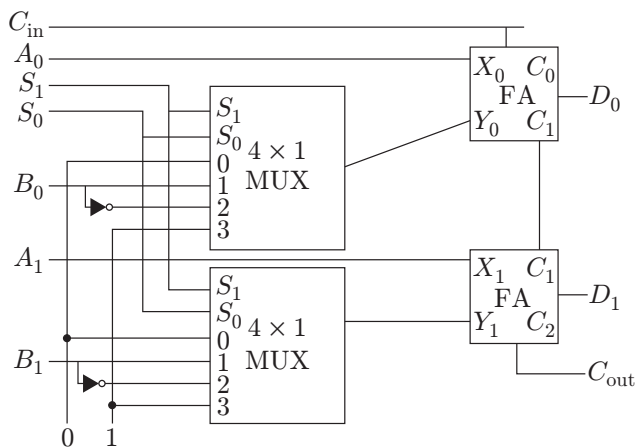
Select			Input to Adder Y	Output of Binary Adder $D = A + Y + C_{in}$	Micro-Operation
S_1	S_0	C_{in}			
0	0	0	0	$D = A$	Transfer A
0	0	1	0	$D = A + 1$	Increment A
0	1	0	B	$D = A + B$	Add
0	1	1	B	$D = A + B + 1$	Add with carry
1	0	0	$\bar{B}$	$D = A + \bar{B}$	Subtract with borrow
1	0	1	$\bar{B}$	$D = A + \bar{B} + 1$	Subtract
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A

2.4 ARITHMETIC LOGIC UNIT

Arithmetic logic unit (ALU) is a combinational circuit that performs all arithmetic and logic operations so that the entire register transfer operation from the source registers through the ALU and into the destination register can be performed during one clock pulse period.

2.4.1 Arithmetic Micro-Operations

The basic arithmetic micro-operations such as addition, subtraction, increment, decrement and shift are performed on numeric data stored in registers. The basic component of arithmetic is parallel binary adder, and by controlling the input to adder, different micro-operations can be realized. Figure 2.1 depicts a 2-bit arithmetic circuit which includes two full-adder circuits and two multiplexers for choosing different arithmetic micro-operations. There are two 2-bit input numbers A and B and 2-bit output D . The two inputs from A go directly to X inputs of full adder. The output of multiplexer goes to input Y of full adder.

**Figure 2.1** | A 2-bit arithmetic circuit.

By controlling the output Y of multiplexers with two selection inputs S_1 and S_0 and C_{in} either 0 or 1, we can generate the eight arithmetic micro-operations (Table 2.2).

2.4.2 Logic Micro-Operations

Logic micro-operations such as AND, OR, Exclusive OR, etc., consider each bit of register separately and specify binary operations for strings of bits (Table 2.3).

Table 2.3 | Types of micro-operations

Micro-operation	Name
$F \leftarrow 0$	Clear
$F \leftarrow A \wedge B$	AND
$F \leftarrow A \wedge \bar{B}$	
$F \leftarrow A$	Transfer A
$F \leftarrow \bar{A} \wedge B$	
$F \leftarrow B$	Transfer B
$F \leftarrow A \oplus B$	Exclusive OR
$F \leftarrow A \vee B$	OR
$F \leftarrow \overline{A \vee B}$	NOR
$F \leftarrow \overline{A \oplus B}$	Exclusive NOR
$F \leftarrow \bar{B}$	Complement B
$F \leftarrow A \vee \bar{B}$	
$F \leftarrow \bar{A}$	Complement A
$F \leftarrow \bar{A} \vee B$	
$F \leftarrow \overline{A \wedge B}$	NAND
$F \leftarrow \text{all 1's}$	Set to all 1's

Logical micro-operations are capable of manipulating individual bits or a portion of word stored in CPU registers. Let us consider the data in a register A . In another register, B is the operand that will be used to modify the contents of A using logic micro-operations. Some of the applications are as follows:

1. **Selective set operation:** In this, If a bit in B is set to 1, that same position in A sets to 1, otherwise that bit in A retains its previous value.

$$\begin{array}{rcl} 1 & 1 & 0 & 0 & A_t \\ 1 & 0 & 1 & 0 & B \text{ (To set some bits in } A) \\ \hline 1 & 1 & 1 & 0 & A_{t+1} (A \leftarrow A + B) \end{array}$$

2. **Selective complement operation:** If a bit in B is set to 1, that same position in A gets complemented from its original value, otherwise it remains unchanged.

$$\begin{array}{rcl} 1 & 1 & 0 & 0 & A_t \\ 1 & 0 & 1 & 0 & B \text{ (To complement some bits in } A) \\ \hline 0 & 1 & 1 & 0 & A_{t+1} (A \leftarrow A \oplus B) \end{array}$$

3. **Selective clear operation:** If a bit in B is set to 1, that same position in A sets to 0, otherwise it remains unchanged.

$$\begin{array}{rcl} 1 & 1 & 0 & 0 & A_t \\ 1 & 0 & 1 & 0 & B \text{ (To clear some bits in } A) \\ \hline 0 & 1 & 0 & 0 & A_{t+1} (A \leftarrow A \cdot B') \end{array}$$

4. **Mask operation:** If a bit in B is set to 0, that same position in A sets to 0, otherwise it remains unchanged.

$$\begin{array}{rcl} 1 & 1 & 0 & 0 & A_t \\ 1 & 0 & 1 & 0 & B \text{ (To clear some bits in } A) \\ \hline 0 & 0 & 0 & 0 & A_{t+1} (A \leftarrow A \cdot B) \end{array}$$

5. **Clear operation:** If the bits in the same position in A and B are the same, they are cleared in A , else they are set in A .

$$\begin{array}{rcl} 1 & 1 & 0 & 0 & A_t \\ 1 & 0 & 1 & 0 & B \\ \hline 0 & 1 & 1 & 0 & A_{t+1} (A \leftarrow A \oplus B) \end{array}$$

6. **Insert operation:** It is used to insert a specific bit pattern into A register, leaving the other bit positions unchanged. This is accomplished by two sub-operations: masking operation to clear the desired bit positions, followed by OR operation to introduce the new bits into the desired positions. Suppose you wanted to introduce 10 into the low order two bits of A :

1101 A (Original) and 1110 A (Desired)

$$\begin{array}{rcl} 1 & 1 & 0 & 1 & A \text{ (Original)} \\ 1 & 1 & 0 & 0 & \text{Mask} \\ 1 & 1 & 0 & 0 & A \text{ (Intermediate)} \\ 0 & 0 & 1 & 0 & \text{Added bits} \\ \hline 1 & 1 & 1 & 0 & A \text{ (Desired)} \end{array}$$

2.5 CPU CONTROL DESIGN

Central processing unit (CPU), or the brain of a computer, performs the data processing operations. It consists of three major parts: register set that stores intermediate data during instruction execution, ALU performs the required micro-operations and control unit that supervises all other elements for the transfer of information from one register to the other. The main function of a CPU is to fetch an instruction from the memory and execute it. CPU is divided into three types of organizations:

1. **Single accumulator organization:** In this, one operand is implied in the accumulator, a special purpose register, and the other operand is a register or the memory. Example: ADD R_1 ($R_1 \leftarrow AC + R_1$), LOAD A ($AC \leftarrow A$), STORE T ($M[T] \leftarrow AC$).
2. **General register organization:** In this, the CPU will have several general purpose registers which lead to shorter and efficient programs because registers are faster. Example: ADD R_1, R_2 ($R_1 \leftarrow R_1 + R_2$). Figure 2.2 shows bus organization for three registers R_1, R_2 and R_3 . The output of these registers and one from the external input is connected to two multiplexers A and B . The two

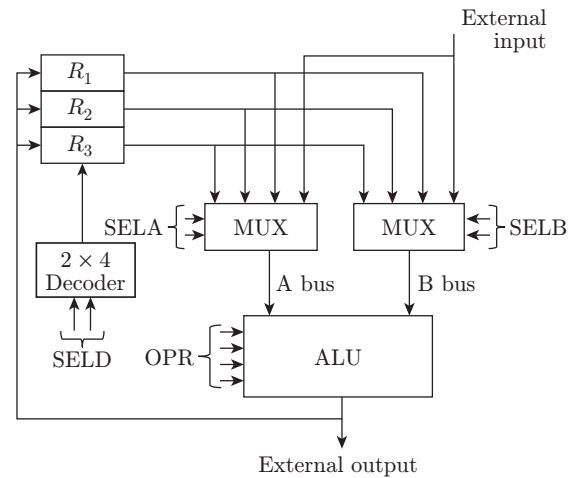


Figure 2.2 | General register organization.

select lines SELA and SELB from multiplexers A and B select one of the input and feed to ALU. OPR specifies one of the possible operation codes that ALU will perform on the data inputs and the output is transferred either to one of the registers using 2×4 decoder or to the external output say memory. The control word (Fig. 2.3) for the two-operand instruction is as follows:

2 bits	2 bits	2 bits	4 bits
SELA	SELB	SELD	OPR

Figure 2.3 | A control word.

3. Stack organization: Stack may consist of number of registers or a part of main memory in which data items are stored in consecutive locations that are accessed by LIFO (last in, first out) mechanism. As there is limited number of registers, a part of memory is implemented as stack for storage and retrieval of intermediate data. Stack pointer (SP) keeps a track of the top item of a stack. The process of inserting a new item onto a stack is known as push accomplished by first incrementing stack pointer and then inserting an item from the data register.

$$SP \leftarrow SP + 1$$

$$M[SP] \leftarrow DR$$

The process of removing an item from the top of a stack is known as pop performed by first transferring data into DR and then decrementing SP.

$$DR \leftarrow M[SP]$$

$$SP \leftarrow SP - 1$$

Problem 2.5: A system has CPU organized in the form of general register organization consisting of 16 registers, each storing 32-bit data. Assume the ALU has 35 operations.

- How many multiplexers are there in A bus and B bus, and what is the size of each multiplexer?
- How many selection inputs are needed for MUX A and MUX B ?
- How many inputs and outputs are there in a decoder?
- How many inputs and outputs are there in ALU for data, including input and output carries?
- Formulate a control word for the system.

Solution:

- 32 Multiplexers, each of size 16×1 .
- 4 Inputs each, to select one of 16 registers.
- 4 to 16 – Line decoder
- $32 + 32 + 1 = 65$ data input lines
- $32 + 1 = 33$ data output lines

4 bits	4 bits	4 bits	6 bits
SELA	SELB	SELD	OPR

2.5.1 Instruction Execution

A CPU generally executes one instruction at a time sequentially and a sequence of such instructions is known as a program. The CPU executes the instructions that reside in the main memory. In order to execute an instruction, the CPU has to fetch the instruction first from the main memory into one of its registers. It then **decodes** the instruction, that is, it decides what the instruction intended to do, fetch operands required and finally **executes** the instruction. This process is

repeated continuously for a complete program and is known as the **fetch–execute** cycle (Fig. 2.4). The following steps are performed for executing an instruction:

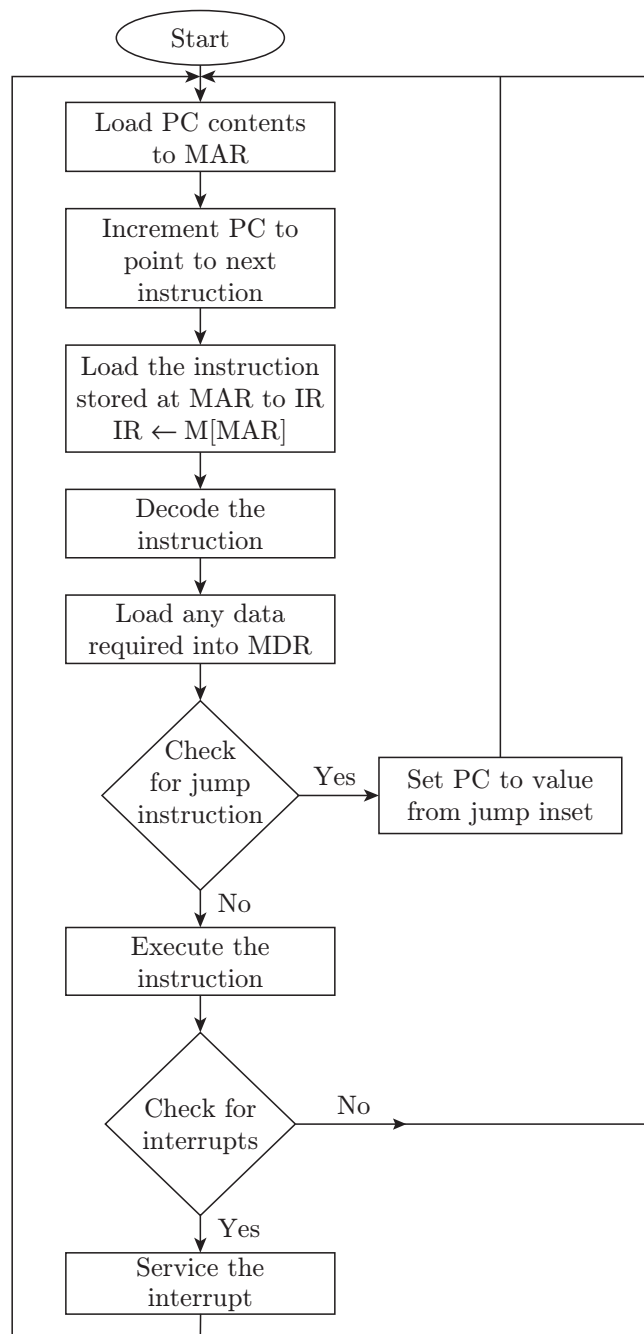


Figure 2.4 | Instruction cycle.

- 1. Fetching the instruction:** The next instruction is fetched from the memory address that is saved in the program counter, and memory content fetched is stored in instruction register (IR). The program counter then points to the next instruction that will be read in the next cycle.
- 2. Decode the instruction:** During this cycle, the instruction inside the IR gets interpreted by the decoder.

3. **Operand fetch:** In case of a direct or indirect memory instruction, the execution begins in the next clock cycle. If the instruction has an indirect address, the effective address of the operand is read from the main memory, and the required data is fetched from the memory into memory data registers. If the instruction has direct address, nothing is done at this clock cycle.
4. **Execute the instruction:** The control unit of the CPU passes the instruction decoded by decoder as a sequence of control signals to the different functional units of the CPU to execute the tasks required by the instruction such as reading values from registers or input devices, performing mathematical or logic micro-operations by ALU, and writing the result back to a register or main memory.

2.5.2 CPU Data Path

CPU contains data paths that are responsible for routing data between the functional units of a computer. The following are the different data path structures available for routing:

1. **Single bus structure:** In this architecture, all CPU registers are connected to the same bus. Data can be transferred either between CPU registers or between CPU register and ALU at a given clock pulse. The speed of operation is slow as only one operand can be transferred in one clock cycle and addition operation ($R_1 \leftarrow R_2 + R_3$) occurs in three clock cycles.
2. **Two bus structure:** All general purpose CPU registers are connected to both buses say bus A and bus B; but special purpose registers are divided into two groups, say group 1 connecting bus A to program counter and one input of ALU and group 2 connecting bus B to MDR (Memory Data Register) and other input of ALU. The two operands are transferred to ALU in 1 clock cycle and the addition operation ($R_1 \leftarrow R_2 + R_3$) occurs in 2 clock cycles.
3. **Three bus structure:** The performance can be further be improved by using three buses such that addition operation ($R_1 \leftarrow R_2 + R_3$) can occur in one clock cycle.

2.5.3 Control Unit Design

Control unit is considered as brain of a CPU that controls various units in the data path. The performance of control unit is important as it determines the clock cycle of the processor. Control unit can be designed either by hardwired or by microprogram.

1. **Hardwired control:** Control unit is made up of sequential and combinatorial circuits to generate the control signals and interpret instructions (Fig. 2.5). The instruction decoder decodes the instruction loaded in instruction register. The step

decoder generates a separate control line for each step in the control sequence. The encoder gets its input signal from the decoder, step decoder, external input and condition codes and generates individual control signals. It is faster and more efficient but less flexible and is difficult to add new feature or correct mistakes in original design.

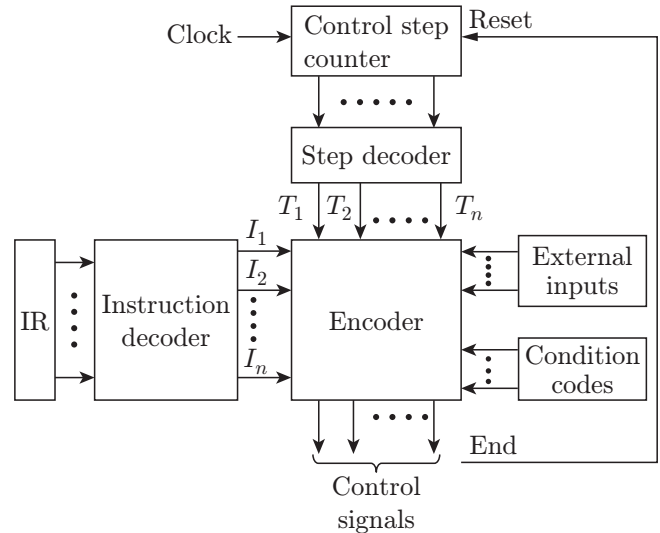


Figure 2.5 | Block diagram of hardwired control unit.

2. **Micro-programmed control:** Control signals are generated by using programming known as micro-programs that constitutes micro-instructions (control word) (Fig. 2.6). Memory that is part

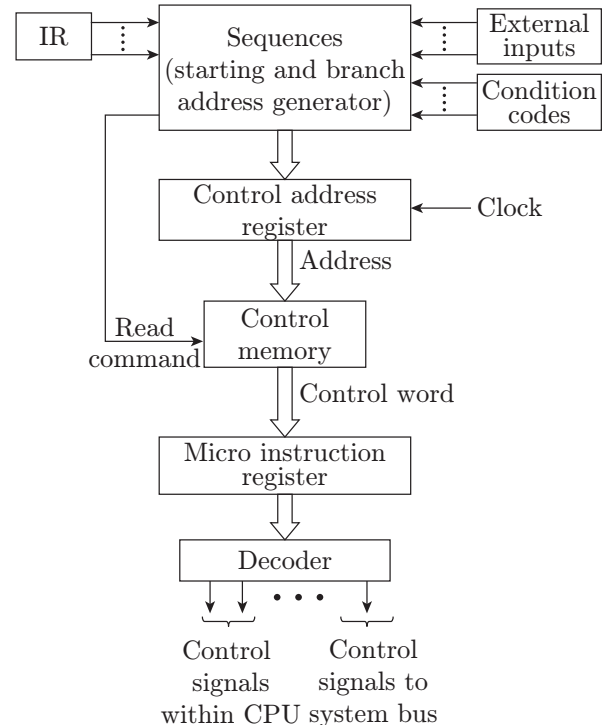


Figure 2.6 | Block diagram of micro-programmed control unit.

of CPU is known as control memory and stores micro-instructions. The micro-program sequencer generates the address of micro-instruction according to instruction stored in instruction register. The address of micro-instruction to be executed is available in content addressable register. Micro-program sequencer issues read command to read micro-instruction from control memory into micro-instruction register which on execution generates control signals for various parts of a processor. This control unit design is more flexible to accommodate new features and less error prone but quite slower than the hardwired unit.

The format of the control word is

Branch condition	Flag	Control signal	Control memory address
------------------	------	----------------	------------------------

On the basis of the type of control word supported, it is divided into two types:

- 1. Horizontal micro-programmed control unit:** In this design, the control signals are represented in the form of 1 bit per control signal and it supports longer control word.
- 2. Vertical micro-programmed control unit:** In this design, the control signal is represented by using encoding format.

Problem 2.6: Consider a control unit which has 1024 control word memory; it supports 48 control signals and 8 flag conditions. What is the size of the control word in bits and control memory in bytes?

Solution:

- (a) Using horizontal programmed control unit

0 bits	3 bits	48 bits	10 bits
Branch condition	Flag	Control signal	Control memory

Size of control word = 61 bits

Control memory = $(1024 \times 61)/8 = 128 \times 61$ bytes

- (b) Using vertical programmed control unit

0 bits	3 bits	48 bits	10 bits
Branch condition	Flag	Control signal	Control memory

log 48 ~ 6 bits

Size of control word = 19 bits

Control memory = $(1024 \times 19)/8 = 128 \times 19$ bytes

Table 2.4 | RISC versus CISC

RISC (Reduced Instruction Set Computers)	CISC (Complex Instruction Set Computers)
Rich register set	Less number of registers
Supports less addressing modes	Supports more number of addressing modes
Supports fixed length instruction	Supports variable length instruction
Successful pipeline with one instruction per cycle	Unsuccessful pipeline
Example: ARM, Motorola	Example: Pentium processors

2.6 I/O INTERFACE (INTERRUPT AND DMA MODE)

I/O interface bridges the differences between CPU and peripheral devices and provides a method for transferring information between internal storage and external I/O devices. There are the following three modes of I/O transfer:

- 1. Programmed I/O:** The I/O device does not have direct access to memory. It requires execution of several instructions by the CPU and the CPU has to wait for the I/O device to be ready for either reception or transmission of data.
- 2. Interrupt initiated I/O:** In this, instead of waiting, the control is transferred from a currently running program to another service program as a result of an external/internal generated request.
 - Hardware interrupts:** These interrupts are present in the hardware pins.
 - Software interrupts:** These are the instructions used in the program whenever the required functionality is needed.
 - Maskable interrupts:** These interrupts may be enabled or disabled explicitly.
 - Non-maskable interrupts:** These interrupts are always there in the enable state. We cannot disable them by explicit conditions (flags).
 - Vectored interrupts:** These interrupts are associated with the static vector address.
 - Non-vectored interrupts:** These interrupts are associated with dynamic vector address.
 - External interrupts:** These interrupts are generated by external devices such as I/O.
 - Internal interrupts:** These devices are generated by the internal components of the processor such as temperature sensor, power failure, error instruction, etc.

2.5.4 RISC versus CISC Processors

The differences between reduced and complex instruction set computers is given in Table 2.4

- **Synchronous interrupts:** These interrupts are controlled by the fixed time interval. All the interval interrupts are called as synchronous interrupt.
- **Asynchronous interrupts:** These interrupts are initiated based on the feedback of previous instructions. All the external interrupts are called as asynchronous interrupt.

3. Direct memory access (DMA): It is one of several methods for coordinating the data transfers between an I/O device and the core processing unit or memory in a computer. It refers to transfer of data directly between a fast storage device and memory bypassing CPU because of its limited speed. DMA provides a significant improvement in terms of latency and throughput as it allows the I/O device to access the memory directly, without using the processor. There are certain advantages of using DMA for data transfer:

- DMA saves processor's MIPS as the core can operate in parallel.
- DMA saves power because it requires less circuitry than the processor to transfer data.
- DMA has no modulo block size restrictions.

Direct memory access (DMA) controller takes over the control of buses to manage the transfer directly between the I/O device and memory. Bus request (BR) and Bus grant (BG) signals are used by the DMA controller to request the CPU to relinquish control of the buses and get the control of system buses (Fig. 2.7). The DMA controller consists of 3 different registers: an address register, a control register and a word counter register. To transfer a block of data between an I/O device and memory, the controller stores initial values in the address register. The DMA channel then transfers the block of information from or to memory according to the control register. The starting address of the

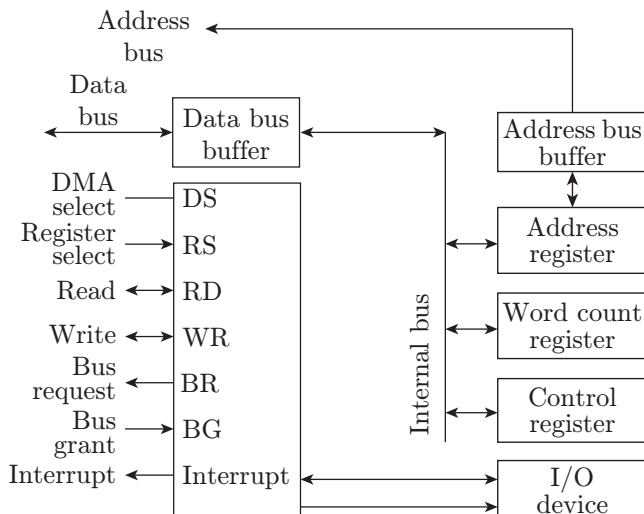


Figure 2.7 | Block diagram of the DMA controller.

block in memory is given by the address register, and the length of the bytes to transfer is given by the word count register. The controller decrements a word counter each time it moves a data byte.

There are several modes of operation of DMA:

- **Burst or block transfer mode:** In this mode, the entire block of data is transferred once the DMA controller is granted access to the system bus by the CPU. The bytes of data in the block are transferred before releasing control of the system buses back to the CPU. The only disadvantage of this mode is that it renders the CPU inactive for some long periods of time.
- **Cycle stealing mode:** In this mode, the DMA controller obtains access to the system buses like burst mode; but after one byte of data transfer, the control of the system bus is released back to the CPU via BG. It is then continually requested again via BR, transferring one byte of data per request, until the entire block of data has been transferred. This mode is suitable for the systems in which the CPU cannot be disabled for the considerable length of time as in burst transfer modes such as for controllers monitoring the data in real time. The advantage is that CPU is not idled for as long as in burst mode, but the data block is not transferred as quickly.
- **Transparent mode:** It is the slowest yet more efficient data transfer mode in terms of overall system performance. The DMA controller transfers data only when the CPU is busy in performing operations that do not use the system buses. So, the CPU never stops executing its programs but the biggest disadvantage is complex hardware circuitry that needs to determine when the CPU is not using the system buses.

A DMA read transfers data from the memory to the I/O device, while DMA write transfers data from an I/O device to memory. The functional behaviour of a DMA transfer outlined in Fig. 2.8:

- The CPU transmits the following information to a DMA controller:
 - (a) beginning address in memory which is stored in address register in DMA controller.
 - (b) Number of words to transfer which is stored in word count register in DMA Controller.
 - (c) direction (memory-to-I/O device or I/O device-to-memory), port ID, DMA mode of transfer and end of block transfer either through interrupt request or no interrupt request which is stored in control register as command word.
- The processor then relinquishes control of address, data and control buses to DMA Controller and returns to other processing activities while the DMA controller starts the data transfer between I/O device and memory.

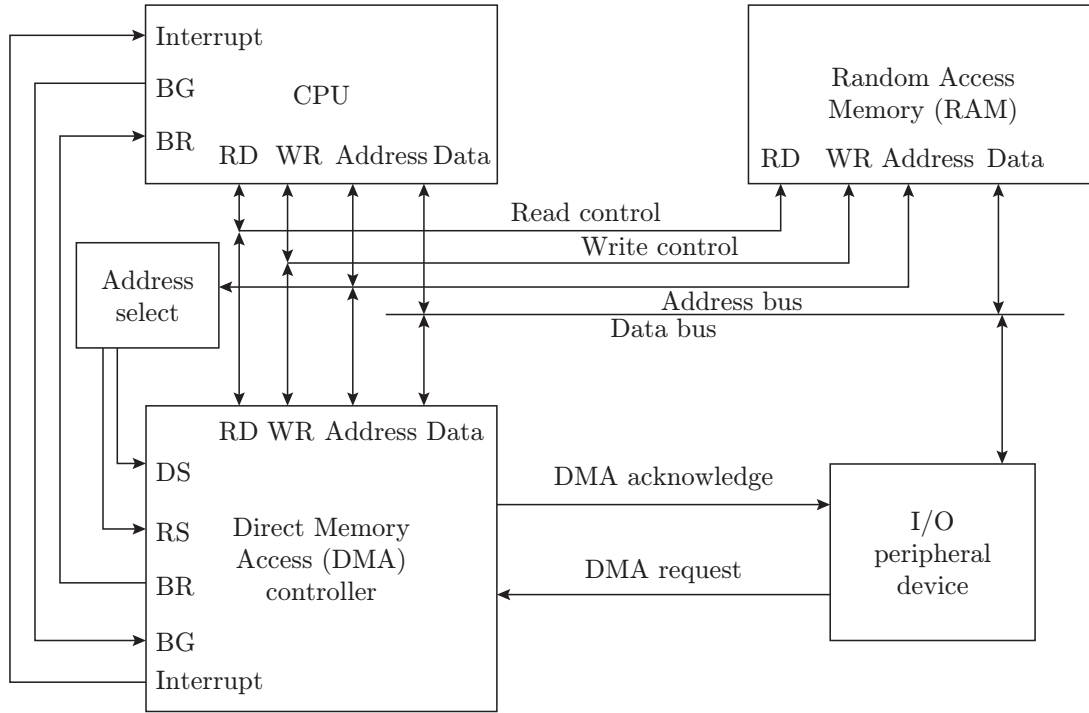


Figure 2.8 | DMA controller interconnection with memory, CPU and I/O devices.

- When the DMA controller accesses memory, it synchronizes this memory request with an idle period of the processor, thus disabling the processor, or requesting a halt of the processor, and awaits an acknowledgement.
- After the completion of the block transfer, the DMA controller either raises an interrupt request if the interrupts are enabled or indicates the completion in its status register and the processor recognizes I/O completion (either by interrupt signal or by reading the status register) and gets its system buses back and normal processing starts. The device has to initiate a new data transfer through DMA request signal which is again acknowledged by CPU through DMA acknowledge signal via DMA controller.

2.7 INSTRUCTION PIPELINING

In early computers, each instruction completely finished before the execution of the next one began. The hardware circuits needed to perform different operations of an instruction cycle are different and most part of these processor circuits are idle at a given moment of time. These processor circuits wait for the other parts of the processor to complete its part of execution first. Instruction pipelining is a technique for overlapping the execution of several instructions to reduce the overall

execution time of a set of instructions and there is no need to wait of the most part of the processor circuits for the other parts of the processor to complete their part of execution. Pipeline speed is limited by the slowest pipeline stage.

Throughput of a processor is the rate at which operations get executed. Latency is the amount of time that a single operation takes to execute. In an unpipelined computer, throughput = $1/\text{latency}$, as each operation executes by itself and for pipelined computer, throughput $> 1/\text{latency}$, since execution of instruction is overlapped.

Consider a k -segment pipeline with a clock cycle time T_p used to execute n tasks (Fig. 2.9). An equivalent non-pipelined system takes T_n time to complete each task. The speed up of a pipelined system over a non-pipelined system is given by the following relation:

$$S = \frac{n \times T_n}{(k + n - 1) \times T_p}$$

Theoretically, maximum speed up that a pipelined system can achieve is given by the following equation:

$$S = \frac{kT_p}{T_n} = k$$

Pipelining Hazards: These hazards reduce the ideal speed up gained by pipelining by preventing the next instruction in the sequence from being executing during its designated clock pulse. Hazards forces the pipeline to be stalled. There are three types of hazards:

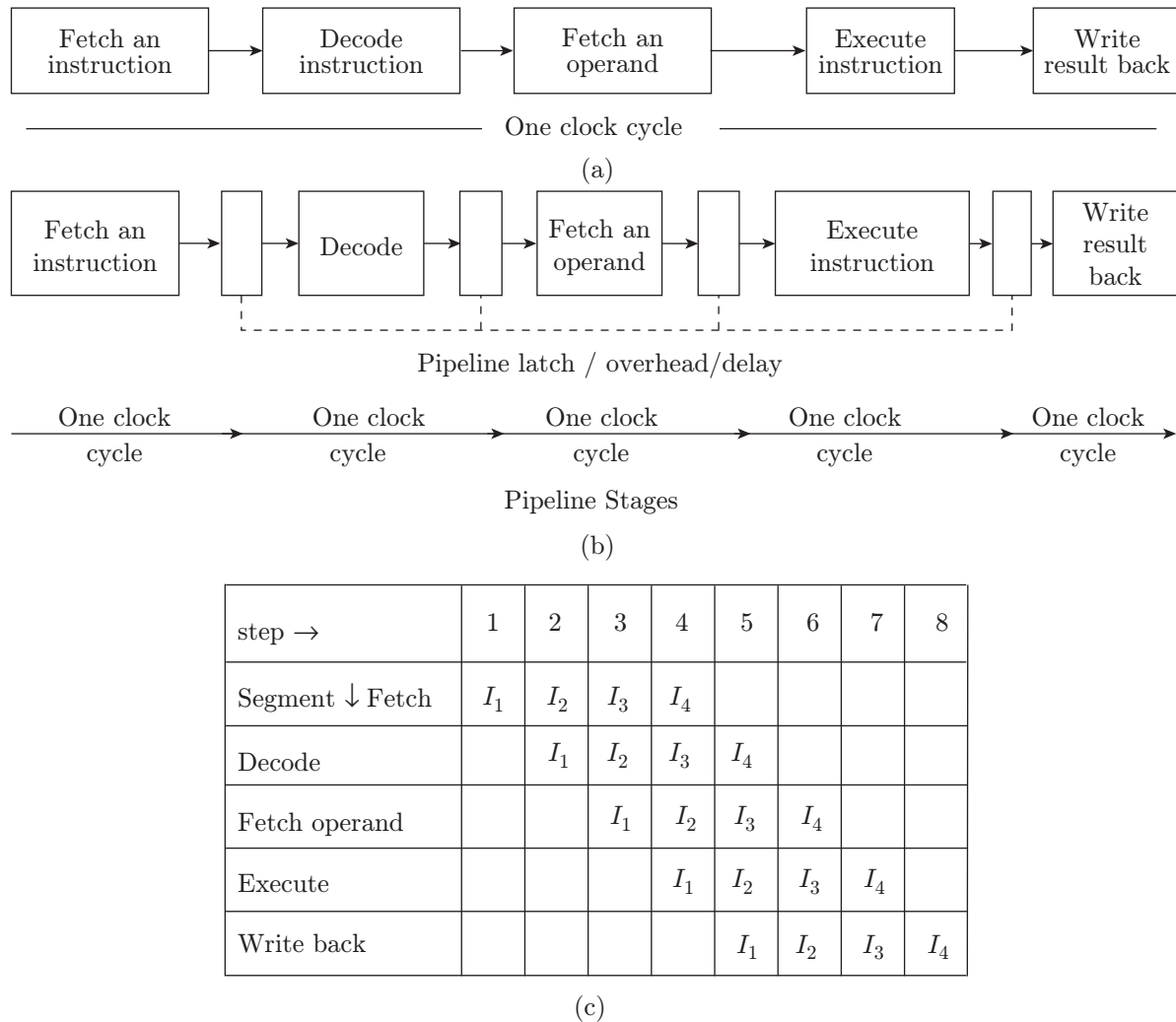


Figure 2.9 | (a) Unpipelined processor. (b) Pipelined five-stage processor.
(c) Timing diagram of a five-stage instruction pipeline.

- 1. Structural hazards:** These result from resource conflicts when the hardware cannot support instructions that need simultaneous execution in pipelining.
- 2. Data hazards:** They arise when an instruction depends on the result of a previous instruction and that result is not yet calculated.

There are three situations in which data hazards can occur:

- Read after write (RAW), a true dependency
- Write after read (WAR), an anti dependency
- Write after write (WAW), an output dependency

Consider two instructions *inst1* and *inst2* occurring, with *inst1* occurring before *inst2* in the program order.

- **Read after write (RAW):** A read after write (RAW) data hazard is a situation in which an

instruction **refers** to a result which is yet not been calculated, that is, in this *inst2* tries to read a source before *inst1* writes to it. This situation arises if the read operation by instruction takes place before write done by other instruction. For example,

```
inst1: R3 <-R1 + R2
inst2: R4 <-R3 + R2
```

The first instruction calculates a value by adding values in registers R1 and R2 and saves the result in register R3, and the second instruction uses this saved value to calculate a result for register R4. However, in a pipeline, when operands for the second operation are fetched, the results from the first instruction will not have been saved yet, and so there arises a data dependency. It can be said that there is a data dependency with instruction *inst2*, as it is dependent on the completion of instruction *inst1*.

- **Write after read (WAR):** A write after read (WAR) data hazard refers to a situation in which there is a problem with concurrent execution, that is, *inst2* tries to write a destination before it is read by *inst1*. This situation arises if write operation completes first by instruction before the read operation takes place by other instruction. For example,

```
inst1: R4 <-R1 + R3
inst2: R3 <-R1 + R2
```

If a situation arises in which there is a chance that *inst2* may get completed before *inst1* (i.e., with concurrent execution) we must note that we do not store the result of register R3 before *inst1* has had a chance to fetch the operands.

- **Write after write (WAW):** A write after write (WAW) data hazard refers to a situation in which there is a concurrent execution environment, that is, *inst2* tries to write an operand before it is written by *inst1*. This situation arises if write operation by an instruction occurs in the reverse order of the intended sequence. For example,

```
inst1: R2 <-R1 + R3
inst2: R2 <-R4 + R5
```

The WB (write back) of *inst2* must be delayed until the execution of *inst1*.

3. **Control hazards:** They arise from the pipelining of branches and other instructions that change the value of PC.

Speed up from pipelining

$$= \frac{\text{Average instruction time unpipelined}}{\text{Average instruction time pipelined}}$$

Speed up from pipelining

$$= \frac{\text{CPI unpipelined} \times \text{Clock cycle pipelined}}{\text{CPI pipelined} \times \text{Clock cycle pipelined}}$$

$$\text{Ideal CPI} = \frac{\text{CPI unpipelined}}{\text{Pipeline depth}}$$

Speed up from pipelining

$$= \frac{\text{Ideal CPI} \times \text{Pipeline depth} \times \text{Clock cycle unpipelined}}{\text{CPI pipelined} \times \text{Clock cycle pipelined}}$$

Speed up from pipelining

$$= \frac{\text{Ideal CPI} \times \text{Pipeline depth} \times \text{Clock cycle unpipelined}}{(\text{Ideal CPI} + \text{Pipeline stall}) \times \text{Clock cycle pipelined}}$$

Assuming ideal CPI as 1, speed up is:

Speed up from pipelining

$$= \frac{\text{Pipeline depth} \times \text{Clock cycle unpipelined}}{(1 + \text{Pipeline stall}) \times \text{Clock cycle pipelined}}$$

where CPI is cycles per instruction.

Problem 2.7: Consider a four-stage pipeline processor. The number of cycles needed by the four instructions I_1 , I_2 , I_3 and I_4 in stages instruction fetch, decode, operand fetch and execute are shown below. Assume I_2 is the branch instruction. Draw the timing space diagram.

	S_1	S_2	S_3	S_4
I_1	2	1	1	1
I_2	1	2	3	1
I_3	1	1	1	2
I_4	2	1	3	1

Solution:

STEP →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Fetch	I_1	I_1	I_2	I_3	—	—	—	—	—	I_3	I_4	I_4					
Decode			I_1	I_2	I_2	—	—	—	—	—	I_3	—	I_4				
Operand Fetch				I_1		I_2	I_2	I_2	—	—	—	I_3	—	I_4	I_4	I_4	
Execute					I_1	—	—	—	I_2	—	—	—	I_3	I_3	—	—	I_4

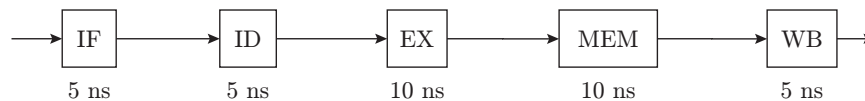
Problem 2.8: Assume a simple 5-stage pipeline (IF, ID, E, DF, W) each stage takes a single cycle. Assuming there are no cache misses. How many cycles would the following code take to execute if there is no special hardware to improve performance in the presence of hazards?

```
MOV    edx,[ecx+100]
MOV    ebx,[ecx+104]
ADD    edx,ebx
MOV    [ecx+108],ebx
MOV    eax,[ecx+100]
ADD    ebx,eax
```

Solution: The above code takes 14 cycles to execute, as shown below:

1	2	3	4	5	6	7	8	9	10	11	12	13	14
IF	ID	DF	E	W									
	IF	ID	DF	E	W								
		IF	ID	DF	stall	E	W						
			IF	ID	stall	DF	stall	W					
				IF	ID	stall	DF	stall	stall	E	W		
					IF	ID	stall	DF	stall	stall	stall	E	W

Problem 2.9: In the below figure, calculate the total execution time after which the result of the fourth task entering the pipe above ready?



Solution:

	5	10	15	20	25	30	35	40	45	50	55	60	65
Inst1	IF	ID	EX	EX	MEM	MEM	WB						
Inst2		IF	ID		EX	EX	MEM	MEM	WB				
Inst3			IF	ID			EX	EX	MEM	MEM	WB		
Inst4				IF	ID				EX	EX	MEM	MEM	WB

Therefore, the total execution time is 65 ns.

Problem 2.10: What is the mean overhead of a pipeline with 8 stages and an execution time per stage of 2 ns?

Solution: The mean overhead = (Stages - 1) × Execution time per stage = (8 - 1) × 2 = 7 × 2 = 14 ns

Problem 2.11: How many stages has a pipeline that achieves a speed of 9.9 for 100 operations?

Solution:

$$\text{Speed} = \frac{n \times k}{k - 1 + n} \Rightarrow 9.9 = \frac{n \times 90}{(90 - 1) + n} \Rightarrow n = 11$$

Problem 2.12: Calculate the time required to perform 1000 operations in a 6-staged pipeline with an execution time of 3 ns per stage?

Solution:

$$T_p = (k - 1 + n) \times T = (6 - 1 + 1000) \times 3 = 3.015 \mu\text{s}$$

Problem 2.13: Calculate the mean overhead of a pipeline with 7 stages and an execution time per stage of 2 ns?

Solution: Mean overhead of pipeline =

$$\frac{(k \times T_p - T_n)}{k} = (k - 1) \times T = (7 - 1) \times 2 = 12 \text{ ns}$$

Problem 2.14: Consider a pipeline with 5 stages: IF, ID, EX, M and W. Assume that each stage requires one clock cycle. Show how the following program segment for adding 2 arrays is processed and compare the clock cycles needed in non-pipelined system with pipelined system when result of the branch instruction i.e. content of is available after WB stage.

```
LOAD R4 #400
L1: LOAD R1, 0 (R4);
LOAD R2, 400 (R4);
ADD R3, R1, R2;
STORE R3, 0 (R4);
SUB R4, R4, #4;
BNEZ R4, L1;
```

Solution: Number of cycles = [Initial instruction + (Number of instructions in the loop L1) × Number of loop cycles] × Number of clock cycles/instruction (CPI)
 $= [1 + (6) \times 400/4] \times 5 = 3005$

Timing diagram for one loop iteration in a pipelined system is as follows:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
LOAD R4 #400	IF	ID	EX	M	W										
LOAD R1, 0 (R4)		IF	ID	EX	M	W									
LOAD R2, 400 (R4)			IF	ID	stall	stall	EX	M	W						
ADD R3, R1, R2				IF	ID	stall	stall	EX	stall	M	W				
STORE R3, 0 (R4)					IF	ID	stall	DF	stall	stall	E	W			
SUB R4,R4, #4							IF	ID	stall	Ex	M	W			
BNEZ R4, L1								IF	stall	ID	stall	stall	EX	M	W

Number of cycles in the loop = 15

Number of clock cycles for segment execution on pipelined processor

$$= 1 + (\text{Number of clock cycles in the loop L1}) \times \text{Number of loop cycles}$$

$$= 1 + 15 \times 400/4 = 1501$$

$$\text{Speedup} = \frac{\text{Number of Clock cycles for the program execution on non-pipelined processor}}{\text{Number of Clock cycles for the segment execution on pipelined processor}}$$

$$= \frac{3005}{1501} = 2 \text{ times}$$

Problem 2.15: Consider a 5-stage pipeline with stages: For all following questions we assume that: (a) Pipeline contains stages: IF (Instruction Fetch), IS (Issue), FO (Fetch operand), E (Execute) and W (Write). (b) Each stage except E requires one clock cycle and system has 4 Functional Units for floating point operations, FP load/store, FP addition/subtraction, FP multiplication and FP division, (c) Execution stage for Load/Store operations requires 1 clock cycle, for ADD or SUB operations requires 1 clock cycle, for MUL operation requires 3 clock cycles and for DIV operation requires 4 clock cycles. All memory references hit in cache. Pipeline has forwarding circuitry for all FUs, except FP-Load/Store where operand is ready after W-stage.

Timing diagram of is presented below:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
LOAD F6, 20(R5)	IF	IS	FO	E	W											
LOAD F2, 28(R5)		IF	ISD	FO	E	W										
MUL F0, F2, F4			IF	IS	stall	stall	FO	E	E	E	W					
SUB F8, F6, F3				IF	IS	FO	E	W								
DIV F10, F0, F6					IF	IS	stall	stall	stall	stall	FO	E	E	E	E	W
ADD F6, F8, F2						IF	IS	FO	E	W						
STORE F8, 50(R5)							IF	IS	FO	E	W					

Identify the hazards in the following instructions from the following list (Structural, Data, Control, RAW, WAR, WAW, None)

1. MULT F0, F2, F4 and STORE F8, 50(R5)
2. DIV F10, F0, F6 and ADD F6, F8, F2
3. MULT F0, F2, F4 and DIV F10, F0, F6
4. DIV F10, F0, F6 and ADD F6, F8, F2

Solution: 1. Structural; 2. Data; 3. RAW; 4. WAR.

2.8 MEMORY HIERARCHY

The storage media can be categorized in hierarchy according to their speed and cost (Fig. 2.10). As we move down the hierarchy, access time increases and cost per bit decreases.

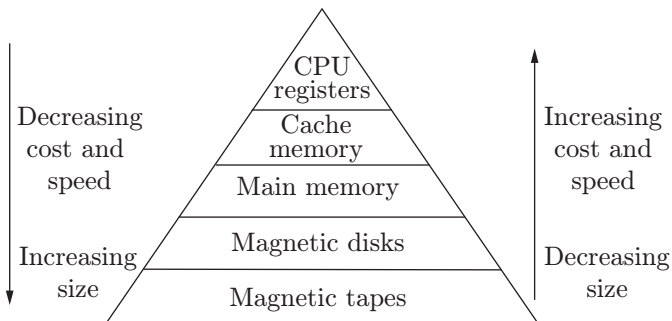


Figure 2.10 | Memory hierarchy.

2.8.1 Main Memory

It is the central storage unit that directly communicates with the CPU. It is designed using semiconductor-integrated circuits and needs constant power supply to maintain the information. It is expensive as compared to auxiliary storage so it has limited capacity. Example: R/W (read/write) memory or RAM (random access memory)

and ROM (read only memory). Integrated RAM chips are available in two modes:

1. **Static RAM:** It stores the binary information in flip flops and information remains valid until power is supplied. It has faster access time and is used in implementing cache memory.
2. **Dynamic RAM:** It stores the binary information as a charge on the capacitor. It requires refreshing circuitry to maintain the charge on the capacitors after few milliseconds. It contains more memory cells per unit area as compared to SRAM.

2.8.1.1 Memory Interfacing

If the required memory for the computer is larger than the capacity of one chip, it is necessary to connect multiple RAM and ROM chips to a CPU through the data and address buses (Fig. 2.11). The low-order address bus lines select the word within a chip and other lines select a particular chip through its chip select inputs. Assume a computer system needs 256 bytes of RAM and 512 bytes of ROM. The configuration of RAM chip is 128×8 and ROM chip is 512×8 . The RAM and ROM chips required are as follows:

$$\text{Number of RAM chips} = 256/128 = 2$$

$$\text{Number of ROM chips} = 512/512 = 1$$

The memory interconnection is depicted in the following diagram:

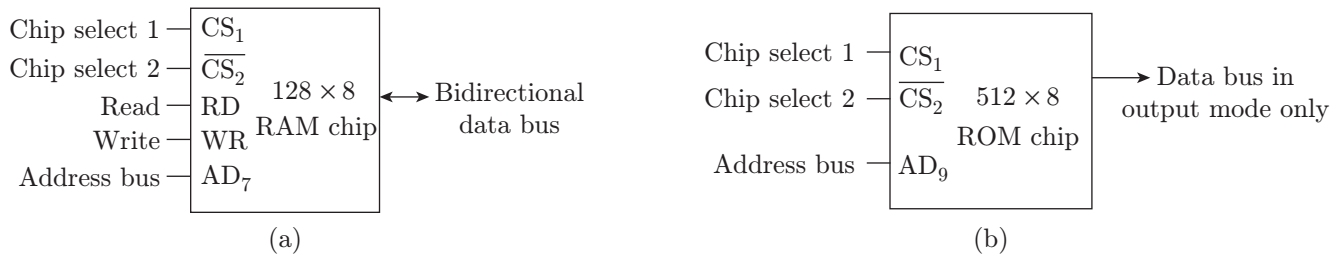


Figure 2.11 | (a) RAM chip. (b) ROM chip.

Problem 2.16: A computer employs RAM chips of 256×8 and ROM chips of 1024×16 . The computer system needs 2K bytes of RAM and 4K bytes of ROM and four interface units each with four registers. Draw a memory address map for the system and give the address range in hexadecimal for RAM and ROM chips.

Solution: RAM $2048/256 = 8$ chips; $2048 = 2^{11}$; $256 = 2^8$

ROM $4096/1024 = 4$ chips; $4096 = 2^{12}$; $1024 = 2^{10}$

Interface $4 \times 4 = 16$ registers; $16 = 2^4$

Component	Address	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
RAM	0000-07FF	0	0	0	0	0	$\leftrightarrow$	3×8 decoder									
ROM	4000-4FFF	0	1	0	0	$\leftrightarrow$	x	2×4 decoder									
Interface	8000-800F	1	0	0	0	0	0	0	0	0	0	0	0	x	x	x	x

2.8.2 Secondary Memory

Secondary memory, also known as auxiliary memory or external memory, can store a large amount of data at lesser cost per byte than the main memory. They are non-volatile in nature, that is, data is not lost when the device is powered off. The most common auxiliary storage devices used in consumer systems are flash memory, optical disks and magnetic disks.

- Flash memory:** Flash memory is an electronic non-volatile fastest computer storage device that can be electrically erased and reprogrammed. Example: flash drives and solid state drive.
- Optical disk:** Optical disks are low-cost mass storage devices from which read and write operations are performed using laser technology. Optical disks can store huge amounts of data up to 6 GB (6 billion bytes). Different types of optical disks are CD-ROM (compact disk read-only), WORM (write-once read-many), EO (erasable optical disks) and DVD.
- Magnetic disk:** A magnetic disk is composed of a circular platter made of metal or plastic and

coated with magnetized material on both sides. Multiple disks are stacked over one another on the spindle with read/write heads on each surface. Bits are stored as spots on magnetized surface along concentric circles called tracks. Tracks are further divided into wedge-shaped sectors.

- Magnetic tapes:** It consists of tape made up of plastic covered with magnetic oxide coating. Tapes are mounted on reels. Bits are recorded as magnetic spots on tape along several tracks. R/W heads are mounted in each track so that data can be recorded and read as a sequence of characters. Seven or nine bits are recorded to form a character together with a parity bit. Data is recorded in contiguous blocks separated by inter-record gaps.

2.8.3 Cache Memory

It is a special memory that compensates the speed mismatch between processor and main memory access time. It temporarily stores frequently used instructions and data for faster processing by the CPU. Cache hit ratio is calculated to measure its performance. If a data

item requested by the CPU is found in cache it is called hit otherwise it is a miss. Hit ratio is defined as ratio of number of hits divided by total CPU references to memory.

$$\text{Hit ratio } (h) = \frac{\text{Number of hits}}{\text{Number of hits} + \text{Number of misses}}$$

$$\text{Average access time} = \text{Hit ratio} \times T_c + (1 - \text{Hit ratio})(T_c + T_m)$$

where T_c is cache access time and T_m is the main memory access time.

2.8.3.1 Elements of Cache Design

The various elements of cache design are as follows:

1. **Cache size:** It should be optimum, small enough to keep average cost per bit close to the main memory and large enough to keep overall average access time close to the cache memory.
2. **Mapping function:** It describes the mapping of main memory block to cache block. There are three different mapping techniques: fully associative, direct mapped and set associative cache organization.
3. **Replacement algorithm:** When a new memory block is required in cache, one of the existing blocks must be replaced by a new block. Example: FIFO (first in, first out), LRU (least recently used).
4. **Write policy:** Cache memory follows write-through and write-back updating policies. In write-through policy, cache controller copies data immediately to main memory as data is written in cache. The data in main memory is always valid, but this approach reduces system performance. In write back, update to memory block is delayed until the updated cache block is replaced by a new block.

2.8.4 Cache Mapping Techniques

The cache memory can store a reasonable number of blocks, but this number is always small as compared to blocks in the main memory to keep average cost per bit low. The correspondence between memory blocks and cache block is specified by the following mapping techniques. Consider a cache memory consisting of 2K words with 128 blocks of 16 words each. Number of bits required to address a cache block is 11 bits. Main memory has 64K words and bits required to address is 16 (Fig. 2.12).

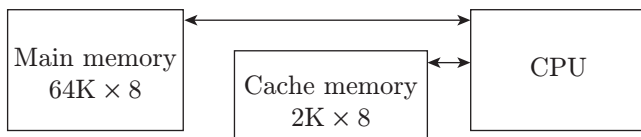


Figure 2.12 | Cache mapping example.

2.8.4.1 Direct Mapping

In this technique, each block from the main memory has only one possible location in the cache memory. In this example, say a block from main memory maps onto a block $(i \bmod 128)$ of the cache. If there are 2^n words in the cache memory and 2^m words in the main memory, then m -bit main memory address is divided into two fields: n bits for index field to access the cache and $(m - n)$ bits for the tag field. Each word in cache consists of the data and the associated tag. Whenever a new block is brought into cache, tag is stored along with data bits. Index field is further divided into block and word if there are multiple words (say k) in a block. The lower k bits select one of the k words in a block known as word field. The block field is used to distinguish a block from other blocks.

Tag $(m - n)$ bits		Index (n) bits
Tag $(m - n)$ bits	Block $(n - k)$ bits	Word (k) bits

When CPU generates a memory request, the block field points to a particular block location in the cache. The high-order tag field is compared with tag bits associated with that cache location. If they match, then the desired word is in that block of cache. If there is no match, then the block containing the required word must be loaded to cache first (Fig. 2.13).

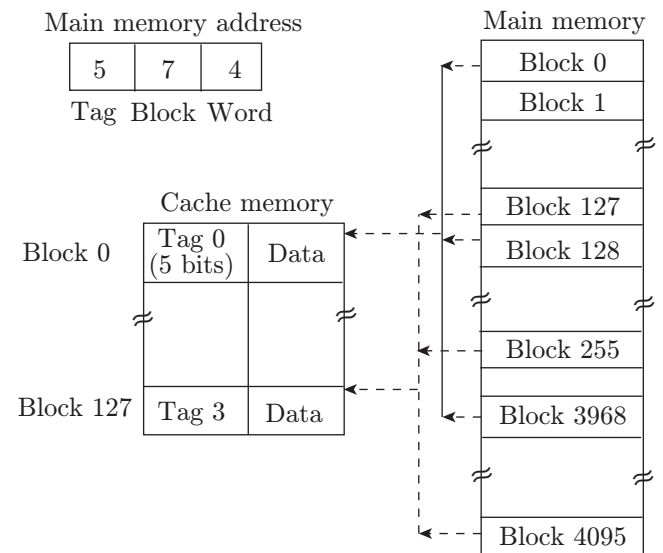


Figure 2.13 | Direct mapped cache organization.

The demerit of direct mapping is that hit ratio drops considerably if two or more words having same index and different tags are accessed consecutively one after the other.

2.8.4.2 Fully Associative Mapping

In this technique, a main memory block can be placed into any cache block location. It is the most flexible cache organization. The main memory address is divided into

two fields: word and tag. The associative memory stores both the address (tag) and data of the main memory. Figure 2.14 shows the mapping of different blocks into cache. High-order 12 bits of CPU address is placed in the argument register of the associative memory and compared to tag bits of each block of the cache to see if the desired block is present. Once the desired block is present, 4-bit word is used to extract necessary word from the cache.

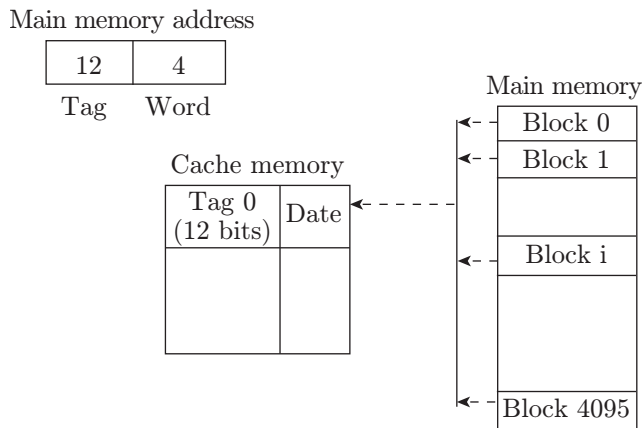


Figure 2.14 | Associative mapped cache organization.

It is necessary to compare high-order bits of main memory with all tag bits corresponding to each block to find whether a given block is present in cache, so it is the most expensive.

2.8.4.3 Set-Associative Mapping

As fully associative mapping is an expensive solution and direct mapping does not allow words with same index but different tag to exist in cache, set associative mapping is a combination of both. It is an improvement over direct mapping where contention problem is solved by having several choices for block placement. The figure below shows two-way set associative cache because each block of main memory has two choices for block placement in cache. A block i in the main memory can be in any block belonging to set $i \bmod S$ of cache, where S is the number of sets. The block 0, 64, 128, ... and so on of main memory can map into any of the two blocks in set 0.

The main memory address is divided into three fields: low-order bits for word field, set field to determine the desired block from all possible sets and high-order bits for the tag field. Each word in cache consists of data and the associated tag.

Tag	Set	Word
-----	-----	------

When the CPU generates a memory request, the set field points to a particular set of the cache which might

contain the desired block. The high-order tag field is then compared associatively to the tags corresponding to the matched set. If a match occurs, the corresponding word is read from cache else main memory is referred and block containing that word is brought into cache for future reference (Fig. 2.15).

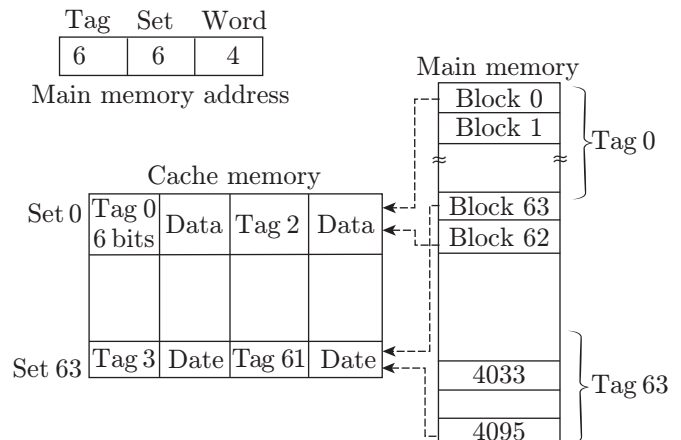


Figure 2.15 | Set-associative mapped cache organization.

Problem 2.17: Consider a memory hierarchy system containing a cache, a main memory and a virtual memory. Assuming, cache access time of 5 ns, and 80% hit ratio. The access time of the main memory is 100 ns, and it has a 99.5% hit rate. The access time of the virtual memory is 10 ms. Calculate the average access time of the memory hierarchy.

Solution: As we know, the hit rate of virtual memory is 100%, the average access time for requests that reach the main memory as $(100 \text{ ns} \times 0.995) + (10 \text{ ns} \times 0.005) = 50,099.5 \text{ ns}$. Given this, the average access time for requests that reach the cache is $(5 \text{ ns} \times 0.80) + (50,099.5 \text{ ns} \times 0.20) = 10,024 \text{ ns}$.

Problem 2.18: A computer uses RAM chips of 1024×1 capacity.

- How many chips are needed to provide a memory capacity of 16K bytes?
- How many of these lines will be common to all chips?

Solution:

- Chips are needed to provide a memory capacity of 16K bytes = $16 \times 8 = 128$ chips
- Using 14 address lines ($16\text{K} = 2^{14}$), we have 10 lines specifying the chip address which is common to all chips.

Problem 2.19: Consider a 2-way set associative cache consisting of 256 blocks of 8 words each, and assume that the main memory is addressable by 16-bit address and it consists of 4K blocks. Calculate the number of bits in each of the TAG, BLOCK/SET and word fields for different mapping techniques?

Solution: For direct mapping, word field is of 3 bits to identify 8 different words in a block ($2^3 = 8$). As cache memory consists of 256 blocks so ($2^8 = 256$) 8 bits are required to address a block because there is one-to-one correspondence of block k in main memory to block $(k \bmod 256)$ in cache memory. The remaining 5 ($16 - 8 - 3$) high-order address bits are tag bits. Thus, the main memory address for direct mapping is divided as follows:

Tag	Block	Word
5 bits	8 bits	3 bits

For fully associative mapping, number of word bits are same, that is, 3 bits. Cache memory stores both tag and data. The high-order tag bits of an address generated by CPU are compared with tag bits of each block so number of block bits is zero. All remaining bits (except word bits) are identified as tag bits. Thus, the main memory address for fully associative mapping is divided as follows:

Tag	Word
13 bits	3 bits

For two-way set-associative mapping, cache memory is mapped with two blocks per set. The word field has same number of bits, 3 bits. There are 128 ($256/2$) sets so 7 bits are required to uniquely identify these 128 sets. The remaining 6 ($16 - 7 - 3$) bits are used as tag bits. Thus, the main memory address for set-associative mapping is divided as follows:

Tag	Set	Word
6 bits	7 bits	3 bits

Problem 2.20: The access time of a cache memory is 200 ns and that of main memory is 2000 ns. It is estimated that 70% of the memory requests are for read and remaining 30% for write. The hit ratio for read accesses only is 0.9. A write-through procedure is used.

- What is the average access time of the system considering only memory read cycles?
- What is the average access time of the system for both read and write requests?
- What is the hit ratio taking into consideration the write cycles also?

Solution:

- Average access time = $0.9 \times 200 + 0.1 \times 2200 = 180 + 220 = 400$ ns
- Average access time = $0.3 \times 2000 + 0.7 \times 400 = 600 + 280 = 880$ ns
- Hit ratio = $0.7 \times 0.9 = 0.63$

Problem 2.21: A 4-way set-associative cache memory uses blocks of four words. The cache can accommodate a total of 1024 words from the main memory. The main memory size is $128K \times 32$.

- Formulate all pertinent information required to construct the cache memory.
- What is the size of the cache memory?

Solution:

- Main memory is $128K = 2^{17}$ so 17 bits address is generated. For a set size of 4, the number of sets in cache is $256/4 = 64$, so 6 bits are required. Each block consists of 4 words so 2 bits are required for the word field.

Tag	Set	Word
9 bits	6 bits	2 bits

- Since cache is 4-way set associative, 4 blocks per set are stored in cache memory.

	Tag 1 9 bits	Data 1 32 bits	Tag 2 9 bits	Data 2 32 bits	Tag 3 9 bits	Data 3 32 bits	Tag 4 9 bits	Data 4 32 bits
Set 0								
-	Tag	Data	Tag	Data	Tag	Data	Tag	Data
Set 128								

Size of cache memory is $128 \times 4(9 + 32) = 5248$ bits.

Problem 2.22: Suppose physical memory is of 2GB and each word is of 16 bits. There is a cache containing 2K words of data, and each cache block contains 16 words. For each of the direct mapped and 2-way set associative cache configurations, specify how the address would be partitioned.

Solution:

- (a) **For direct mapping**, the word field is of 4 bits identify 16 different words in a block ($2^4 = 16$). As the cache memory consists of 2K words which is equivalent to $2K/16 = 128$ blocks so ($2^7 = 128$) 7 bits are required to address a block. The remaining 20 ($31 - 7 - 4$) high-order address bits are tag bits. Thus, the main memory address for direct mapping is divided as follows:

20 bits	7 bits	4 bits
Tag	Block	Word

- (b) Since cache is 2-way set associative, 2 blocks per set are stored in cache memory. So, the number of sets is $128/2 = 64$.

21 bits	6 bits	4 bits
Tag	Set	Word

Problem 2.23: Consider a direct mapped cache of size 32 KB with block size 32 bytes. The CPU generates 32-bit addresses. What are number of bits needed for addressing block in cache and number of tag bits?

Solution:

Tag	Block	Word
$32 - 10 - 5$ bits	10 bits	5 bits

The number of bits needed for addressing block in cache and number of tag bits are 10, 17, respectively.

IMPORTANT FORMULAS

- Amdahl's law

$$S_{\text{overall}} = \left[(1 - \sum F_i) + \sum \frac{F_i}{S} \right]^{-1}$$

- The speed up of pipelined system over non-pipelined system is given by:

$$S = \frac{n \times T_n}{(k + n - 1) \times T_p}$$

- Maximum speed up that a pipelined system can achieve is given by:

$$S = \frac{kT_p}{T_n} = k$$

- Hit ratio (h)

$$= \frac{\text{Number of hits}}{\text{Number of hits} + \text{Number of misses}}$$

- Average access time

$$= \text{Hit ratio} \times T_c + (1 - \text{Hit ratio})(T_c + T_m)$$

where T_c is cache access time and T_m is main memory access time.

- Direct mapping

Tag ($m - n$) bits	Block ($n - k$) bits	Word (k bits)
----------------------	------------------------	------------------

- Set-associative mapping

Tag	Set	Word
-----	-----	------

SOLVED EXAMPLES

- The principle of locality is used in

- (a) Interrupt (b) Registers
(c) DMA (d) Cache memory

Solution: It is used in cache memory to help the program access small amounts of address space at any instant.

Ans. (d)

- Which memory unit has lowest access time?

- (a) Cache (b) Registers
(c) Optical disk (d) Main memory

Solution: Registers are used for processing and manipulating data and for holding memory addresses that are available to the machine-code programmer. So, they have lowest access time.

Ans. (b)

3. During DMA transfer, the DMA controller takes over the buses to manage the transfer
- (a) Directly from CPU to memory
 - (b) Directly from memory to CPU
 - (c) Directly between the memory and registers
 - (d) Directly between the I/O device and memory

Solution: DMA controller manages transfer between I/O device and memory.

Ans. (d)

4. Booth's algorithm is used for the arithmetic operation of

- (a) addition.
- (b) subtraction.
- (c) multiplication.
- (d) division.

Solution: It is a multiplication algorithm that multiplies two signed binary numbers in 2's complement notation.

Ans. (c)

5. The reason for improvement in CPU performance during pipelining is

- (a) reduced memory access time.
- (b) increased clock speed.
- (c) introduction of parallelism.
- (d) increase in cache memory.

Solution: Instruction-level parallelism is implemented within a single processor to allow faster CPU throughput.

Ans. (c)

6. Use of cache memory enhances

- (a) I/O access time
- (b) memory access time.
- (c) effective memory access time.
- (d) secondary storage access time.

Solution: Cache memory compensates the speed mismatch between processor and main memory access time.

Ans. (c)

7. An instruction cycle refers to

- (a) fetching an instruction.
- (b) executing an instruction.
- (c) fetching, decoding and executing an instruction.
- (d) reading and executing an instruction.

Solution: It involves fetching, decoding and executing the instruction.

Ans. (c)

8. A hardware interrupt is also called

- (a) an internal interrupt.
- (b) an external interrupt.

- (c) a processor interrupt.
- (d) a clock interrupt.

Solution: Hardware interrupt is present in hardware pins.

Ans. (b)

9. Priority is provided by _____ for access to memory by various I/O channels and processors.

- (a) a register
- (b) a counter
- (c) the processor scheduler
- (d) a controller

Solution: Controller sets priority for memory access by various I/O devices and processes.

Ans. (d)

10. By applying the principle of temporal locality, processes are likely to reference pages that _____

- (a) have been referenced recently.
- (b) are located at address near recently referenced pages in memory.
- (c) have been preloaded into memory.
- (d) have to be reloaded into memory.

Solution: Temporal locality refers to reuse of resources referenced within a short time frame.

Ans. (a)

11. Which of the following is a correct statement related to L2 cache memory?

- (a) The level 1 cache is always faster than the level 2 cache.
- (b) The level 2 cache is used to mitigate the dynamic slowdown every time a level 1 cache miss occurs.
- (c) Level 2 cache comes as on board only.
- (d) In modern day computer, the level 2 cache is considered an internal cache.

Solution: L2 level of cache is placed between the L1 and RAM. The L1 cache is always the fastest.

Ans. (a)

12. What is the control unit's function in the CPU?

- (a) To decode program instructions
- (b) To transfer data to primary storage
- (c) To perform logical operations
- (d) To store arithmetic operations

Solution: Control unit controls several units of CPU and helps decode program instructions.

Ans. (a)

13. CPU fetches the data and instructions from _____

- (a) ROM
- (b) control unit
- (c) RAM
- (d) coprocessors chip

Solution: The CPU fetches data and instructions from RAM and executes it.

Ans. (c)

14. Which of the following affects the processing power of the CPU?

- (a) Data bus, addressing schemes
- (b) Clock speed, addressing schemes
- (c) Clock speed, data bus
- (d) Clock speed, data bus, addressing schemes

Solution: CPU processing speed is affected by clock cycle, data bus for routing data and addressing modes.

Ans. (d)

15. The contents of the flag register after execution of the following program by 8085 microprocessor will be:

Program
SUB A
MVI B, (01)_H
DCR B
HLT

- (a) (54)_H (b) (00)_H (c) (01)_H (d) (45)_H

Solution:

SUB A Subtract contents of memory location whose contents are stored in A
MVI B, (01)_H Move B with (01)_H
DCR B Decrement B gives (54)_H
HLT

Ans. (a)

16. An N -bit carry look-ahead adder, where N is a multiple of 4, employs ICS 74181 (40-bit ALU) and 74182 (4-bit carry look-ahead generator).

The minimum addition time using the best architecture for adder is

- (a) Proportional to N
- (b) Proportional to $\log N$
- (c) A constant
- (d) None of the above

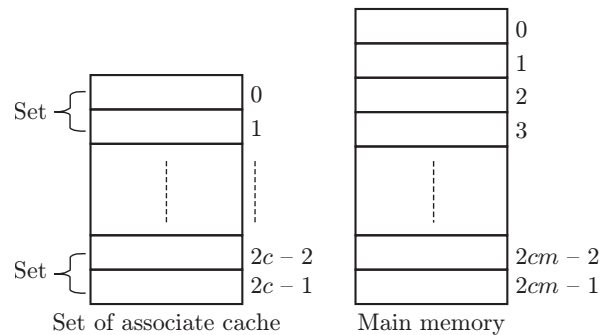
Solution: The addition time for N -bit carry look-ahead adder is always a constant.

Ans. (c)

17. The main memory of a computer has $2cm$ blocks while the cache has $2c$ blocks. If the cache uses the set-associative mapping scheme with two blocks per set, then block k of the main memory maps to the set:

- (a) $(k \bmod m)$ of the cache
- (b) $(k \bmod c)$ of the cache
- (c) $(k \bmod 2c)$ of the cache
- (d) $(k \bmod 2cm)$ of the cache

Solution:



Number of sets in cache

$$= \frac{\text{Number of blocks in cache}}{\text{Number of blocks in one set}} = \frac{2c}{2}$$

Number of sets in cache = c

So to map block k , main memory maps to the set $(k \bmod c)$ of the cache.

Ans. (b)

18. Which of the following is/are advantage of virtual memory?

- (a) Faster memory to memory on an average
- (b) Processes can be given protected address spaces
- (c) Linker can assign addresses independent of where the program will be loaded in physical memory
- (d) Programs larger than the physical memory size can be run

Solution: Virtual memory allows programs to have a larger space than the physical memory size to run. So, the programmer does not have to worry about physical memory size.

Ans. (d)

19. The number of full and half adders required to add 16-bit numbers is

- (a) 8 half adders, 8 full adders
- (b) 1 half adder, 15 full adders
- (c) 16 half adders, 0 full adder
- (d) 4 half adders, 12 full adders

Solution: To add N -bit numbers:

Number of half adders required = 1

Number of full adders required = $N - 1$

Therefore, to add 16-bit number only, 1 half adder and 15 full adders are required.

Ans. (b)

20. Which of the following requires a device driver

- (a) Register
- (b) Cache
- (c) Main memory
- (d) Disk

Solution: Disk is the I/O device attached externally to the processor. Therefore, disk requires a device driver.

Ans. (d)

21. More than one word are put in one cache block to
- (a) exploit the temporal locality of reference in a program.
 - (b) exploit the spatial locality of reference in a program.
 - (c) reduce the miss penalty.
 - (d) none of the above.

Solution: There are two types of locality of references temporal and spatial locality.

The concept of spatial locality, instead of fetching just one item from the main memory to the cache, is useful to fetch several items that reside at adjacent address as well.

So, option (b) is correct.

Ans. (b)

22. Which of the following statements is false?
- (a) Virtual memory implements the translation of a program's address space into physical memory address space.
 - (b) Virtual memory allows each program to exceed the size of the primary memory.
 - (c) Virtual memory increases the degree of multiprogramming.
 - (d) Virtual memory reduces the context-switching overhead.

Solution: Virtual memory increases the context-switching overhead.

Ans. (d)

23. Advantage of synchronous sequential circuits over asynchronous ones is

- (a) faster operation.
- (b) ease of avoiding problems due to hazards.
- (c) lower hardware requirement.
- (d) better noise immunity.

Solution: Because of less delay, synchronous sequential circuits have faster operation than asynchronous ones.

Ans. (a)

24. The total size of address space in a virtual memory system is limited by

- (a) the length of MAR.
- (b) the available secondary storage.
- (c) the available main memory.
- (d) all of the above.

Solution: Virtual memory depends only on the available size of the secondary memory.

Ans. (b)

25. Comparing the time T_1 taken for a single instruction on a pipelined CPU with time T_2 taken on a non-pipelined but identical CPU, we can say that

- (a) $T_1 \leq T_2$
- (b) $T_1 \geq T_2$
- (c) $T_1 < T_2$
- (d) T_1 plus T_2 is the time taken for one instruction fetch cycle

Solution: In case of one instruction, non-pipelined CPU takes less time as compared to pipelined CPU. This is due to buffer delays for pipelining.

Ans. (b)

GATE PREVIOUS YEARS' QUESTIONS

1. For a pipelined CPU with a single ALU, consider the following situations:

- I. The $j + 1$ -st instruction uses the result of the j th instruction as an operand.
- II. The execution of a conditional jump instruction.
- III. The j -th and $j + 1$ -st instructions require the ALU at the same time.

Which of the above can cause a hazard?

- (a) I and II only
- (b) II and III only
- (c) III only
- (d) All the three

(GATE 2003: 1 Mark)

Solution: All the three statements cause hazards.

Ans. (d)

Common Data Questions 2 and 3: Consider the following assembly language program for a hypothetical processor. A , B and C are 8-bit registers. The meanings of various instructions are shown as comments:

```
MOV B, #0;    B ← 0
MOV C, #8;    C ← 8
Z: CMP C, #0; Compare C with 0
JZ X;         Jump to X if zero flag is set
```

SUB $C, \#1$; $C \leftarrow C - 1$
 RRC $A, \#1$; Right rotate A through carry
 by one bit. Thus:
 ; if the initial values of A and
 the carry flag are $a_7..a_0$ and; c_0
 respectively, their values after the
 execution of this; instruction will
 be $c_0 a_7..a_1$ and a_0 , respectively.
 JC Y ; Jump to Y if carry flag is set
 JMP Z ; Jump to Z
 Y: ADD $B, \#1$; $B \leftarrow B + 1$
 JMP Z ; Jump to Z
 X:

2. If the initial value of register A is A_0 , the value of register B after the program execution will be

- (a) The number of 0 bits in A_0
 (b) The number of 1 bits in A_0
 (c) A_0
 (d) 8

(GATE 2003: 2 Marks)

Solution: After execution, register B will contain number of 0 bits in A_0 .

Ans. (a)

3. Which of the following instructions when inserted at location X will ensure that the value of register A after program execution is the same as its initial value?

- (a) RRC $A, \#1$
 (b) NOP; no operation
 (c) LRC $A, \#1$; left rotate A through carry flag by one bit
 (d) ADD $A, \#1$

(GATE 2003: 2 Marks)

Solution: RRC $A, \#1$

Ans. (a)

4. Which of the following addressing modes are suitable for program relocation at run time?

- (i) Absolute addressing (ii) Base addressing
 (iii) Relative addressing (iv) Indirect addressing
 (a) (i) and (iv) (b) (i) and (ii)
 (c) (ii) and (iii) (d) (i), (ii) and (iv)

(GATE 2004: 1 Mark)

Solution: Both base addressing and relative addressing modes are suitable.

Ans. (c)

Common Data Questions 5 and 6: Consider the following program segment for a hypothetical CPU having three-user registers R_1 , R_2 and R_3 .

Instruction	Operation	Instruction Size (in words)
MOV $R_1, 5000$	; $R_1 \leftarrow \text{Memory } [5000]$	2
MOV $R_2 (R_1)$	; $R_2 \leftarrow \text{Memory } [(R_1)]$	1
ADD R_2, R_3	; $R_2 \leftarrow R_2 + R_3$	1
MOV 6000, R_2	; Memory $[6000] \leftarrow R_2$	2
HALT	; Machine halts	1

5. Consider that the memory is byte addressable with size 32 bits, and the program has been loaded starting from memory location 1000 (decimal). If an interrupt occurs while the CPU has been halted after executing the HALT instruction, the return address (in decimal) saved in the stack will be

- (a) 1007 (b) 1020
 (c) 1024 (d) 1028

(GATE 2004: 2 Marks)

Solution:

MOV $R_1, 5000$	; 2	1000 to 1007
MOV $R_2 (R_1)$	; 1	1008 to 1011
ADD R_2, R_3	; 1	1012 to 1015
MOV 6000, R_2	; 2	1016 to 1023
HALT	; 1	1024 to 1027

Ans. (c)

6. Let the clock cycles required from various operations be as follows:

Register to/from memory transfer: 3 clock cycles
 ADD with both operands in register: 1 clock cycle
 Instruction fetches and decodes: 2 clock cycles per word
 The total number of clock cycles required to execute the program is

- (a) 29 (b) 24
 (c) 23 (d) 20

(GATE 2004: 2 Marks)

Solution:

Clock Cycles		
MOV R_1 , 5000	; 2	8
MOV R_2 (R1)	; 1	5
ADD R_2 , R_3	; 1	1
MOV 6000, R	; 2	8
HALT	; 1	2

Total 24 clock cycles are required.

Ans. (b)

7. Consider a small two-way set-associative cache memory, consisting of four blocks. For choosing the block to be replaced, use the least recently used (LRU) scheme. The number of cache misses for the following sequence of block addresses 8, 12, 0, 12, 8 is

(a) 2 (b) 3 (c) 4 (d) 5

(GATE 2004: 2 Marks)

Solution: Sequence is: 8, 12, 0, 12, 8

	12	12	12
8	8	0	8

Total number of miss = 4

Ans. (c)

8. A hard disk with a transfer rate of 10 MB/s is constantly transferring data to memory using DMA. The processor runs at 600 MHz, and takes 300 and 900 clock cycles to initiate and complete DMA transfer, respectively. If the size of the transfer is 20 KB, what is the percentage of processor time consumed for the transfer operation?

(a) 5.0% (b) 1.0% (c) 0.5% (d) 0.1%

(GATE 2004: 2 Marks)

Solution: Given that data transfer rate = 10 MB/s and size of transfer = 20 KB

So, % processor time consumed = $20 \text{ KB} \times 100 / 10 \text{ MB/s} = 0.1\%$

Ans. (d)

9. A 4-stage pipeline has the stage delays as 150, 120, 160 and 140 ns, respectively. Registers that are used between the stages have a delay of 5 ns each. Assuming constant clocking rate, the total time taken to process 1000 data items on this pipeline will be

(a) 120.4 μs (b) 160.5 μs (c) 165.5 μs (d) 590.0 μs

(GATE 2004: 2 Marks)

Solution: Maximum stage delay = $160 + 5 = 165 \text{ ns}$
 $T = [4 + (1000 - 1)] 165 = 165.5 \mu\text{s}$

Ans. (c)

10. Consider a multiplexer with X and Y as data inputs and Z as control input. $Z = 0$ selects input X, and $Z = 1$ selects input Y. What are the connections required to realize the 2-variable Boolean function $f = T + R$, without using any additional hardware?

(a) R to X, 1 to Y, T to Z
 (b) T to X, R to Y, T to Z
 (c) T to X, R to Y, 0 to Z
 (d) R to X, 0 to Y, T to Z

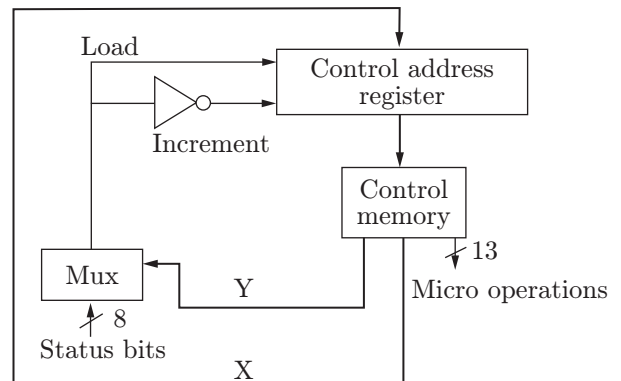
Solution: Connect R to X, 1 to Y and T to Z.

Ans. (a)

11. The microinstructions stored in the control memory of a processor have a width of 26 bits. Each microinstruction is divided into three fields: a micro-operation field of 13 bits, a next address field (X), and a MUX select field (Y). there are 8 status bits in the inputs of the MUX. How many bits are there in the X and Y fields, and what is the size of the control memory in number of words?

(a) 10, 3, 1024 (b) 8, 5, 256
 (c) 5, 8, 2048 (d) 10, 3, 512

(GATE 2004: 2 Marks)



Solution: MUX has 8 input bits, 3 select lines are required. So, $Y = 3$. Total bits = 26

$X = 26 - 13 - 3 = 10$. So, memory required = $2^{10} = 1024$

Ans: (a)

12. Which one of the following is true for a CPU having a single interrupt request line and a single interrupt grant line?

(a) Neither vectored interrupt nor multiple interrupting devices are possible.
 (b) Vectored interrupts are not possible but multiple interrupting devices are possible.
 (c) Vectored interrupts and multiple interrupting devices are both possible.
 (d) Vectored interrupt is possible but multiple interrupting devices are not possible.

(GATE 2005: 1 Mark)

Solution: Vectored interrupts are not possible but multiple interrupting devices are possible.

Ans. (b)

13. Normally user programs are prevented from handling I/O directly by I/O instructions in them. For CPUs having explicit I/O instructions, such I/O protection is ensured by having the I/O instructions privileged. In a CPU with memory mapped I/O, there is no explicit I/O instruction. Which one of the following is true for a CPU with memory mapped I/O?

- (a) I/O protection is ensured by operating system routine(s).
 (b) I/O protection is ensured by a hardware trap.
 (c) I/O protection is ensured during system configuration.
 (d) I/O protection is not possible.

(GATE 2005: 1 Mark)

Solution: I/O protection is ensured by operating system routine(s).

Ans. (a)

14. Consider a three word machine instruction ADD A[R0], @B

The first operand (destination) "A [R0]" uses indexed addressing mode with R0 as the index register. The second operand (source) "@B" uses indirect addressing mode. A and B are memory addresses residing at the second and the third words, respectively. The first word of the instruction specifies the opcode, the index register designation and the source and destination addressing modes.

During execution of ADD instruction, the two operands are added and stored in the destination (first operand).

The number of memory cycles needed during the execution cycle of the instruction is:

- (a) 3 (b) 4
 (c) 5 (d) 6

(GATE 2005: 2 Marks)

Solution: A[R0] = require 1 memory cycle

The 3rd and 4th operands are indirectly addressed so each requires 2 cycles = 4 memory cycles.

1 memory cycle will be required to store the result. So, total of 6 memory cycles are required.

Ans. (d)

15. Match each of the high level language statements given on the left-hand side with the most natural addressing mode from those listed on the right-hand side.

(1) A[I] = B[J]	(a) Indirect addressing
(2) while (*A++)	(b) Indexed addressing
(3) int temp = *x	(c) Auto increment

- (a) (1, c), (2, b), (3, a) (b) (1, a), (2, c), (3, b)
 (c) (1, b), (2, c), (3, a) (d) (1, a), (2, b), (3, c)

(GATE 2005: 2 Marks)

Solution: A[I] = indexed addressing

*A++ = auto increment

Temp = *x = indirect addressing

Ans. (c)

16. Consider a direct mapped cache of size 32 KB with block size 32 bytes. The CPU generates 32-bit addresses. The number of bits needed for cache indexing and the number of tag bits are, respectively

- (a) 10, 17 (b) 10, 22 (c) 15, 17 (d) 5, 17

(GATE 2005: 2 Marks)

Solution: Cache size = 32 KB = 15 bits are required

Index bits = 15

Tag bits 32 - 15 = 17

Ans. (a)

17. A 5-stage pipelined CPU has the following sequence of stages:

IF: Instruction fetch from instruction memory

RD: Instruction decodes and register read

EX: Execute ALU operation for data and address computation

MA: Data memory access for write access, the register read at RD state is used.

WB: Register write back.

Consider the following sequence of instructions:

I_1 : L R₀, loc 1; R₀ $\leftarrow$ M[loc1]

I_2 : A R₀, R₀ 1; R₀ $\leftarrow$ R₀ + R₀

I_3 : S R₂, R₀ 1; R₂ $\leftarrow$ R₂ - R₀

Let each stage take one clock cycle. What is the number of clock cycles taken to complete the above sequence of instructions starting from the fetch of I_1 ?

- (a) 8 (b) 10 (c) 12 (d) 15

(GATE 2005: 2 Marks)

Solution: 12 clock cycles are taken, as shown below

Clocks	1	2	3	4	5	6	7	8	9	10	11	12
I_1	IF	RD	EX	MA	WB							
I_2		IF	—	—	—	RD	EX	MA	WB			
I_3			IF	—	—	—	RD	—	—	EX	MA	WB

Ans. (c)

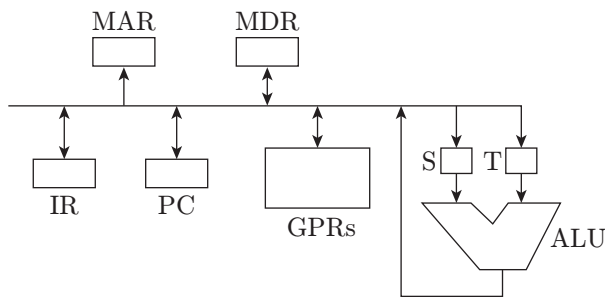
18. A device with data transfer rate 10 KB/s is connected to a CPU. Data is transferred byte-wise. Let the interrupt overhead be 4 s. The byte transfer time between the device interfaces register and CPU or memory is negligible. What is the minimum performance gain of operating the device under interrupt mode over operating it under program-controlled mode?
- (a) 15 (b) 25
(c) 35 (d) 45

(GATE 2005: 2 Marks)

Solution: Data transfer rate = 10 KB/s
 Interrupt overhead = 4×10^{-2} s
 10 KB is sent = 1 s
 1 B is sent = $1/10K = 100 \times 10^{-2}$ s
 Minimum performance gain = $100 \times 10^{-2} / 4 \times 10^{-2}$
 = 25

Ans. (b)

Common Data Questions 19 and 20: Consider the following data path of a CPU. The ALU, the bus and all the registers in the data path are of identical size. All operations including incrementation of the PC and the GPRs are to be carried out in the ALU. Two clock cycles are needed for memory read operation – the first one for loading address in the MAR and the next one for loading data from the memory bus into the MDR.



19. The instruction “add R_0, R_1 ” has the register transfer interpretation $R_0 \leftarrow R_0 + R_1$. The minimum number of clock cycles needed for execution cycle of this instruction is
- (a) 2 (b) 3
(c) 4 (d) 5

(GATE 2005: 2 Marks)

Solution: There will be three cycles—(1) R_{1out}, S_{in} , (2) R_{2out}, T_{in} and (3) $S_{out}, T_{out}, ALU_{add}, R_{in}$.

Ans. (b)

20. The instruction “call R_n , sub” is a two-word instruction. Assuming that PC is incremented during the

fetch cycle of the first word of the instruction, its register transfer interpretation is

$$R_n \leftarrow PC + 1;$$

$$PC \leftarrow M[PC];$$

The minimum number of CPU clock cycles needed during the execution cycle of this instruction is

- (a) 2 (b) 3 (c) 4 (d) 5

(GATE 2005: 2 Marks)

Solution: The minimum number of CPU clock cycles needed during the execution cycle = 4. This is because

1 cycle is required to transfer already incremented value of PC

2 cycles for getting data in MDR

1 to load value of MDR in PC

Ans. (c)

21. Consider a disk drive with the following specifications:

16 surfaces, 512 tracks/surface, 512 sectors/track, 1 KB/sector, rotation speed 3000 rpm. The disk is operated in cycle stealing mode whereby whenever one 4 byte word is ready it is sent to memory; similarly, for writing, the disk interface reads a 4 byte word from the memory in each DMA cycle. Memory cycle time is 40 nsec. The maximum percentage of time that the CPU gets blocked during DMA operation is:

- (a) 10 (b) 25
(c) 40 (d) 50

(GATE 2005: 2 Marks)

Solution:

Data transfer in one rotation = 512×1024 Bytes

$$1 \text{ rotation takes} = \frac{60}{3000} \text{ s}$$

$$512 \text{ KB is transferred in} = \frac{60}{3000} \text{ s}$$

$$1 \text{ byte will be transferred} = \frac{60}{3000} \times 512 \times 1024$$

4 bytes will be transferred

$$= 60 \times \frac{4}{3000} \times 512 \times 1024 = 152.58 \text{ ns}$$

$$\text{Block \%} = \frac{40}{152.28} = 26\%$$

Ans. (b)

22. A CPU has 24-bit instructions. A program starts at address 300 (in decimal). Which one of the following is a legal program counter (all values in decimal)?

- (a) 400 (b) 500
(c) 600 (d) 700

(GATE 2006: 1 Mark)

Solution: Size of instruction = 24 bits; Start address = 300. Legal address will be multiple of three, that is, 300.

Ans. (c)

23. A CPU has a cache with block size 64 bytes. The main memory has k banks, each bank being c bytes wide. Consecutive c byte chunks are mapped on consecutive banks with wrap around. All the k banks can be accessed in parallel, but two accesses to the same bank must be serialized. A cache block access may involve multiple iterations of parallel bank accesses depending on the amount of data obtained by accessing all the k banks in parallel. Each iteration requires decoding the bank numbers to be accessed in parallel and this takes $k/2$ ns. The latency of one bank access is 80 ns. If $c = 2$ and $k = 24$, the latency of retrieving a cache block starting at address zero from the main memory is

- (a) 92 ns (b) 104 ns
(c) 172 ns (d) 184 ns

(GATE 2006: 2 Marks)

Solution:

Time for one parallel process = $k/2 + \text{latency}$

Time for one byte = $24/2 + 80 = 92$

Total time for c bytes = $2 \times 92 = 184$ ns

Ans. (d)

24. A CPU has a five-stage pipeline and runs at 1 GHz frequency. Instruction fetch happens in the first stage of the pipeline. A conditional branch instruction computes the target address and evaluates the condition in the third stage of the pipeline. The processor stops fetching new instructions following a conditional branch until the branch outcome is known. A program executes 10^9 instructions out of which 20% are conditional branches. If each instruction takes one cycle to complete on average, the total execution time of the program is

- (a) 1.0 s (b) 1.2 s
(c) 1.4 s (d) 1.6 s

(GATE 2006: 2 Marks)

Solution:

Total execution time of the program

$$= 10^9 + 0.20 \times 2 \times 10^9 = 1.4 \text{ s}$$

Ans. (c)

25. Consider a new instruction named branch-on-bit-set (mnemonic bbs). The instruction "bbs reg, pos, label" jumps to label if bit in position pos of register operand reg is one. A register is 32 bits wide

and the bits are numbered 0 to 31, bit in position 0 being the least significant. Consider the following emulation of this instruction on a processor that does not have bbs implemented.

temp $\leftarrow$ reg & mask

Branch to label if temp is non-zero.

The variable temp is a temporary register. For correct emulation, the variable mask must be generated by

- (a) mask $\leftarrow 0 \times 1 \ll \text{pos}$
(b) mask $\leftarrow 0 \times \text{ffffffff} \gg \text{pos}$
(c) mask $\leftarrow \text{pos}$
(d) mask $\leftarrow 0 \times f$

(GATE 2006: 2 Marks)

Solution: As there is only one bit with pos, the other bits need to be set to 0 in temp. The mask register must have 1 in pos position, for which pos number of left shifts over 1 need to be made.

Ans. (a)

Common Data Questions 26 and 27: Consider two cache organizations. The first one is 32 KB, 2-way set associative with 32-byte block size. The second one is of the same size but direct mapped. The size of an address is 32 bits in both cases. A 2-to-1 multiplexer has a latency of 0.6 ns while a k -bit comparator has a latency of $k/10$ ns. The hit latency of the set-associative organization is h_1 while that of the direct mapped one is h_2 .

26. The value of h_1 is

- (a) 2.4 ns (b) 2.3 ns
(c) 1.8 ns (d) 1.7 ns

(GATE 2006: 2 Marks)

Solution:

Address bits = 32

Block size = 32 B = 5 bits

Size of cache = 32KB/32 = 1KB = 10 bits

For 2-way set-associative memory = index bits = 9, tag bits = 18

$k/10 = 18/10 = 1.8 + \text{latency} = 1.8 + 0.6 = 2.4$

Ans. (a)

27. The value of h_2 is

- (a) 2.4 ns (b) 2.3 ns
(c) 1.8 ns (d) 1.7 ns

(GATE 2006: 2 Marks)

Solution:

In direct memory access: Tag bits = 17; Index = 10 bits; word = 5 bits

$k/10 = 17/10 = 1.7 + \text{latency} = 1.7 + 0.6 = 2.3$

Ans. (b)

Common Data Questions 28 and 29: A CPU has a 32-KB direct-mapped cache with 128-byte block size. Suppose A is a two-dimensional array of size 512×512 with elements that occupy 8 bytes each. Consider the following two C code segments, P_1 and P_2 .

```
P1: for (i=0; i<512; i++) {
        for (j=0; j<512; j++) {
            x +=A[i][j];
        }
    }
P2: for (i=0; i<512; i++) {
        for (j=0; j<512; j++) {
            x +=A[j][i];
        }
    }
```

P_1 and P_2 are executed independently with the same initial state, namely, the array A is not in the cache and i, j, x are in registers. Let the number of cache misses experienced by P_1 be M_1 and that for P_2 be M_2 .

28. The value of M_1 is

- (a) 0 (b) 2048
(c) 16384 (d) 262144

(GATE 2006: 2 Marks)

Solution:

Memory = 32 KB

Block size = 128 B

Number of blocks = 256

Number of elements in block = $256/8 = 16$

P_1 cache misses = $M_1 : 512 \times 512/16 = 16384$

Ans. (c)

29. The value of the ratio M_1/M_2 is

- (a) 0 (b) $1/16$
(c) $1/8$ (d) 16

(GATE 2006: 2 Marks)

Solution:

P_2 number of cache misses = $M_2 = 512 \times 512$

Ratio of $M_1:M_2 = 1:16$

Ans. (b)

30. Consider a 4-way set-associative cache consisting of 128 lines with a line size of 64 words. The CPU generates a 20-bit address of a word in main memory. The number of bits in the TAG, LINE and WORD fields are, respectively:

- (a) 9, 6, 5 (b) 7, 7, 6
(c) 7, 5, 8 (d) 9, 5, 6

(GATE 2007: 1 Mark)

Solution: For 64 words, $\log_2 64 = 6$ bits are required.
For lines = $128/4 = 32$ lines, 5 bits are required

Tag bits	Line	Word
9	5	6

Ans. (d)

31. Consider a pipelined processor with the following four stages:

IF: Instruction fetch

ID: Instruction decode and operand fetch

EX: Execute

WB: Write back

The IF, ID and WB stages take one clock cycle each to complete the operation. The number of clock cycles for the EX stage depends on the instruction. The ADD and SUB instructions need 1 clock cycle and the MUL instruction needs 3 clock cycles in the EX stage. Operand forwarding is used in the pipelined processor. What is the number of clock cycles taken to complete the following sequence of instructions?

ADD $R_2, R_1, R_0 \quad R_2 \leftarrow R_1 + R_0$

MUL $R_4, R_3, R_2 \quad R_4 \leftarrow R_3 \times R_2$

SUB $R_6, R_5, R_4 \quad R_6 \leftarrow R_5 - R_4$

- (a) 7 (b) 8 (c) 10 (d) 14

(GATE 2007: 1 Mark)

Solution:

Clock	1	2	3	4	5	6	7	8
I_1	IF	ID	EX	WB				
I_2		IF	ID	EX	EX	EX	WB	
I_3			IF	ID	—	—	EX	WB

Using operand forwarding, 8 clock cycles are required.

Ans. (b)

32. In a simplified computer, the instructions are

OP R_j, R_i — Performs R_j OP R_i and stores the result in register R_i .

OP m, R_i — Performs val OP R_i and stores the result in R_i . val denotes the content of memory location m .

MOV, mR_i — Moves the content of memory location m to register R_i .

MOV R_i, m — Moves the content of register R_i to memory location m .

The computer has only two registers, and OP is either ADD or SUB. Consider the following basic block:

$$\begin{aligned} t_1 &= a + b \\ t_2 &= c + d \\ t_3 &= e - t_2 \\ t_4 &= t_1 - t_3 \end{aligned}$$

Assume that all operands are initially in memory. The final value of the computation should be in memory. What is the minimum number of MOV instructions in the code generated for this basic block?

- (a) 2 (b) 3
(c) 5 (d) 6

(GATE 2007: 1 Mark)

Solution: The instructions generated in the code for this basic block are as follows:

```
MOV a, Ri
ADD b, Ri
MOV c, Rj
ADD d, Rj
SUB e, Rj
SUB Ri, Rj
MOV m, Ri
```

Ans. (b)

Common Data Questions 33–35: Consider the following program segment. Here R_1 , R_2 and R_3 are general purpose registers.

Instruction	Operation	Instruction Size (No. of Words)
MOV R_1 , (3000)	$R_1 \leftarrow m[3000]$	2
LOOP: MOV R_2 , (R_3)	$R_2 \leftarrow M[R_3]$	1
ADD R_2 , R_1	$R_2 \leftarrow R_1 + R_2$	1
MOV (R_3), R_2	$M[R_3] \leftarrow R_2$	1
INC R_3	$R_3 \leftarrow R_3 + 1$	1
DEC R_1	$R_1 \leftarrow R_1 - 1$	1
BNZ LOOP	Branch on not zero	2
HALT	Stop	1

Assume that the content of memory location 3000 is 10 and the content of the register R_3 is 2000. The content of each of the memory locations from 2000 to 2010 is 100. The program is loaded from the memory location 1000. All the numbers are in decimal.

33. Assume that the memory is word addressable. The number of memory references for accessing the data in executing the program completely is

- (a) 10 (b) 11
(c) 20 (d) 21

(GATE 2007: 1 Mark)

Solution: Given that $R_1 = 10$, so the loop will run 10 times

$$10 \times 2 + 1 = 21$$

Ans. (d)

34. Assume that the memory is word addressable. After the execution of this program, the content of memory location 2010 is

- (a) 100 (b) 101
(c) 102 (d) 110

(GATE 2007: 1 Mark)

Solution: It will remain 100, because the loop will exit as the value in R_1 becomes 0 when address in R_3 becomes 2010.

Ans. (a)

35. Assume that the memory is byte addressable and the word size is 32 bits. If an interrupt occurs during the execution of the instruction "INC R_3 ", what return address will be pushed on to the stack?

- (a) 1005 (b) 1020
(c) 1024 (d) 1040

(GATE 2007: 1 Mark)

Solution: Memory is byte addressable, take 4 bytes per word. So at INC R_3 , stack will contain 1024.

Ans. (c)

36. Consider a disk pack with 16 surfaces, 128 tracks per surface and 256 sectors per track. 512 bytes of data are stored in a bit serial manner in a sector. The capacity of the disk pack and the number of bits required to specify a particular sector in the disk are respectively:

- (a) 256 Mbyte, 19 bits (b) 256 Mbyte, 28 bits
(c) 512 Mbyte, 20 bits (d) 64 Gbyte, 28 bits

(GATE 2007: 1 Mark)

Solution:

Disk capacity = 16 surfaces $\times$ 128 tracks $\times$

$$256 \text{ sectors} \times 512 \text{ bytes} = 256 \text{ MB}$$

$$\text{Total number of sectors} = 16 \times 128 \times 256 = 2^{19}$$

Ans. (a)

Linked Answer Questions 37 and 38: Consider a machine with a byte addressable main memory of 162 bytes. Assume that a direct mapped data cache consisting of 32 lines of 64 bytes each is used in the system. A 50×50 two-dimensional array of bytes is stored in the main memory starting from memory location 1100H. Assume that the data cache is initially empty. The complete array is accessed twice. Assume that the contents of the data cache do not change in between the two accesses.

37. How many data cache misses will occur in total?

- (a) 48 (b) 50 (c) 56 (d) 59

(GATE 2007: 2 Marks)

Solution:

Main memory = 2^{16} B

Block size = 64 B

Number of blocks = 32

Number of elements = $50 \times 50 = 2500$

Starting from location 1100 means from 68th block

Number of blocks = $2500/64 = 40$ blocks

Initially cache is empty for 32 misses, then 8 are remaining from total 40 for one access

Array is traversed twice, so data cache misses = $40 + 8 + 8 = 56$

Ans. (c)

38. Which of the following lines of the data cache will be replaced by new blocks in accessing the array for the second time?

- (a) line 4 to line 11 (b) line 4 to line 12
(c) line 0 to line 7 (d) line 0 to line 8

Solution:

Applying $k \bmod c$ to find the location: $68 \bmod 32 = 4$ to 11

Ans. (a)

39. Which of the following is/are true for the auto-increment addressing mode?

- I. It is useful in creating self-relocating code.
- II. If it is included in an Instruction Set Architecture, then an additional ALU is required for effective address calculation.
- III. The amount of increment depends on the size of the data item accessed.

- (a) I only (b) II only
(c) III only (d) II and III only

(GATE 2008: 2 Marks)

Solution: Only statement (III) is true.

Ans. (c)

40. Which of the following must be true for the RFE (Return from Exception) instruction on a general purpose processor?

- I. It must be a trap instruction.
- II. It must be a privileged instruction.
- III. An exception cannot be allowed to occur during execution of an RFE instruction.

- (a) I only (b) II only
(c) I and II only (d) I, II and III only

(GATE 2008: 2 Marks)

Solution: RFE (Return from Exception) is a privileged trap instruction that is executed when

exception occurs, so an exception is not allowed to execute. Option (d) is the correct option.

Ans. (d)

41. For a magnetic disk with concentric circular tracks, the seek latency is not linearly proportional to the seek distance due to

- (a) non-uniform distribution of requests
- (b) arm starting and stopping inertia
- (c) higher capacity of tracks on the periphery of the platter
- (d) use of unfair arm scheduling policies

(GATE 2008: 2 Marks)

Solution: Tracks on magnetic disks are concentric and seek latency from one sector to other which may or may not be in different tracks. This seek distance is not proportional to latency since the tracks at periphery have higher diameter, and hence higher capacity to store data.

Ans. (b)

42. Which of the following are NOT true in a pipelined processor?

- I. Bypassing can handle all RAW hazards.
- II. Register renaming can eliminate all register carried WAR hazards.
- III. Control hazard penalties can be eliminated by dynamic branch prediction.

- (a) I and II only (b) I and III only
(c) II and III only (d) I, II and III

(GATE 2008: 2 Marks)

Solution: All the statements are true.

Ans. (d)

43. For inclusion to hold between two cache levels L1 and L2 in a multi-level cache hierarchy, which of the following are necessary?

- I. L1 must be a write-through cache
- II. L2 must be a write-through cache
- III. The associativity of L2 must be greater than that of L1
- IV. The L2 cache must be at least as large as the L1 cache

- (a) IV only (b) I and IV only
(c) I, II and IV only (d) I, II, III and IV

(GATE 2008: 2 Marks)

Solution: L1 and L2 cache are placed between CPU and they can be both write through cache but not necessarily.

Associativity does not matter.

L2 cache must be at least as large as L1 cache, since all the words in L1 are also in L2.

Ans. (a)

44. The use of multiple register windows with overlap causes a reduction in the number of memory accesses for

I. Function locals and parameters
 II. Register saves and restores
 III. Instruction fetches

(a) I only (b) II only (c) III only (d) I, II and III

(GATE 2008: 2 Marks)

Solution: Multiple register windows with overlap causes a reduction in the number of memory accesses for register saves and restores.

Ans. (b)

45. Consider a machine with a 2-way set-associative data cache of size 64 KB and block size 16 bytes. The cache is managed using 32 bit virtual addresses and the page size is 4 KB. A program to be run on this machine begins as follows:

```
double ARR [1024] [1024];
int i, j;
/* Initialize array ARR to 0.0 */
for(i=0; i<1024; i++)
    for(j=0; j<1024; j++)
        ARR [i] [j] =0.0;
```

The size of double is 8 bytes. Array ARR is located in memory starting at the beginning of virtual page 0xFF000 and stored in row major order. The cache is initially empty and no pre-fetching is done. The only data memory references made by the program are those to array ARR.

The total size of the tags in the cache directory is

- (a) 32 Kbits (b) 34 Kbits
 (c) 64 Kbits (d) 68 Kbits

(GATE 2008: 2 Marks)

Solution:

Virtual address = 32 bits

2-way cache size = 64 KB

1 set will contain = 32 KB entries = 15 bits

Block size = 16 bytes = 4 bits

Tag bits	Set bits	Word bits
17	11	4

Tag size = $17 \times 2 \times 1024 = 34$ kbits

Ans. (b)

46. Consider the data given in the above question. Which of the following array elements has the same cache index as ARR[0][0]?

- (a) ARR[0][4] (b) ARR[4][0]
 (c) ARR[0][5] (d) ARR[5][0]

(GATE 2008: 2 Marks)

Solution: Total elements can come in one slot 2048. After 2048 elements, same cache index will be on [2][0] and [4][0].

Ans. (b)

47. The cache hit ratio for this initialization loop is

- (a) 0% (b) 25% (c) 50% (d) 75%

(GATE 2008: 2 Marks)

Solution: Cache hit ratio is found out as follows:

$$\frac{1024}{2048} = \frac{1}{2} = 50\%$$

As we can see in the above, there will be 50% hits.

Ans. (c)

Linked Answer Questions 48 and 49: Delayed branching can help in the handling of control hazards.

48. For all delayed conditional branch instructions, irrespective of whether the condition evaluates to true or false:

- (a) The instruction following the conditional branch instruction in memory is executed.
 (b) The first instruction in the fall through path is executed.
 (c) The first instruction in the taken path is executed.
 (d) The branch takes longer to execute than any other instruction.

(GATE 2008: 2 Marks)

Solution: The first instruction following the branch instruction is always executed (irrespective of whether the branch is taken or not).

Ans. (b)

49. The following code is to run on a pipelined processor with one branch delay slot:

I_1 : ADD $R_2 \leftarrow R_7 + R_8$

I_2 : SUB $R_4 \leftarrow R_5 - R_6$

I_3 : ADD $R_1 \leftarrow R_2 + R_3$

I_4 : STORE Memory $[R_4] \leftarrow R_1$

BRANCH to Label if $R_1 == 0$

Which of the instructions I_1 , I_2 , I_3 or I_4 can legitimately occupy the delay slot without any other program modification?

- (a) I_1 (b) I_2 (c) I_3 (d) I_4

(GATE 2008: 2 Marks)

Solution: Instruction I_2 contains delayed slot. I_4 has data dependency in I_2 .

Ans. (b)

50. How many $32\text{K} \times 1$ RAM chips are needed to provide a memory capacity of 256 K bytes?

(a) 8 (b) 32
(c) 64 (d) 128

Solution:

$$\text{Number of chips required} = \frac{256\text{K} \times 8}{32\text{K} \times 1} = 64$$

Ans. (c)

51. A CPU generally handles an interrupt by executing an interrupt service routine

(a) as soon as an interrupt is raised.
(b) by checking the interrupt register at the end of the fetch cycle.
(c) by checking the interrupt register after finishing the execution of the current instruction.
(d) by checking the interrupt register at fixed time intervals.

Solution: Interrupts are handled by checking the interrupt register after finishing the execution of current instruction.

Ans. (c)

52. Consider a 4-stage pipeline processor. The number of cycles needed by the four instructions I_1, I_2, I_3, I_4 in stages S_1, S_2, S_3, S_4 is shown below:

	S_1	S_2	S_3	S_4
I_1	2	1	1	1
I_2	1	3	2	2
I_3	1	1	1	3
I_4	1	2	2	2

What is the number of cycles needed to execute the following loop?

for (i=1 to 2) { $I_1; I_2; I_3; I_4;$ }

(a) 16 (b) 23 (c) 28 (d) 30

(GATE 2009: 2 Marks)

Solution:

Clock	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
I_1	S_1	S_1	S_2	S_3	S_4										
I_2			S_1	S_2	S_2	S_2	S_3	S_3	S_4	S_4					
I_3				S_1	S_1		S_2		S_3		S_4	S_4	S_4		
I_4							S_1		S_2	S_2	S_3	S_3		S_4	S_4

For two iterations = $15 \times 2 = 30$ clock cycles

Ans. (d)

53. Consider a 4-way set-associative cache (initially empty) with total 16 cache blocks. The main memory consists of 256 blocks and the request for memory blocks is in the following order:

0, 255, 1, 4, 3, 8, 133, 159, 216, 129, 63, 8, 48, 32, 73, 92, 155

Which one of the following memory block will NOT be in cache if LRU replacement policy is used?

(a) 3 (b) 8 (c) 129 (d) 216

(GATE 2009: 2 Marks)

Solution:

To decide the location (address), mod 4 is applied.

Set 0	0, 4, 8, 216 $\rightarrow$ 48, 32, 8, 92
Set 1	1, 133, 129, 73
Set 2	155
Set 3	255, 3, 159, 63

216 will not be there in cache.

Ans. (d)

Common Data Questions 54 and 55: A hard disk has 63 sectors per track, 10 platters each with 2 recording surfaces and 1000 cylinders. The address of a sector is given as a triple $\langle c, h, s \rangle$, where c is the cylinder number, h is the surface number and s is the sector number. Thus, the 0th sector is addressed as $\langle 0, 0, 0 \rangle$, the 1st sector as $\langle 0, 0, 1 \rangle$, and so on.

(GATE 2009: 2 Marks)

54. The address $\langle 400, 16, 29 \rangle$ corresponds to sector number:

(a) 505035 (b) 505036
(c) 505037 (d) 505038

Solution:

Total surfaces = $10 \times 2 = 20$

Address is: $400 \times 20 \times 63 + 16 \times 63 + 29 = 505037$

Ans. (c)

55. The address of 1039th sector is

(a) $\langle 0, 15, 31 \rangle$ (b) $\langle 0, 16, 30 \rangle$
(c) $\langle 0, 16, 31 \rangle$ (d) $\langle 0, 17, 31 \rangle$

Solution:

Address of 1039th sector = $16 \times 31 + 31 = 1039$

Ans. (c)

56. A main memory unit with a capacity of 4 megabytes is built using $1\text{M} \times 1\text{-bit}$ DRAM chips. Each DRAM chip has 1 K rows of cells with 1K cells in each row. The time taken for a single refresh operation is 100 ns. The time required to perform one refresh operation on all the cells in the memory unit is

- (a) 100 nanoseconds
 (b) 100×2^{10} nanoseconds
 (c) 100×2^{20} nanoseconds
 (d) 3200×2^{20} nanoseconds

(GATE 2010: 1 Mark)

Solution:

Main memory = 4 MB

Number of DRAM chips = $4\text{ MB} / 1\text{M} \times 1\text{ bit} = 32$

Total cells = $32 \times 1\text{K} \times 1\text{K}$

Time taken to refresh all the cells = $32 \times 1\text{K} \times 1\text{K} \times 100\text{ ns}$

Ans. (d)

57. The weight of a sequence $a_0, a_1/2, \dots, a_{n-1}$ of real numbers is defined as $a_0 + a_1/2 + \dots + a_{n-1}/2^{n-1}$. A subsequence of a sequence is obtained by deleting some elements from the sequence, keeping the order of the remaining elements the same. Let X denote the maximum possible weight of a subsequence of $a_0, a_1, \dots, a_{n-1}$ and Y the maximum possible weight of a subsequence of $a_1, a_2, \dots, a_{n-1}$. Then X is equal to

- (a) $\max(Y, a_0 + Y)$ (b) $\max(Y, a_0 + Y/2)$
 (c) $\max(Y, a_0 + 2Y)$ (d) $a_0 + Y/2$

(GATE 2010: 2 Marks)

Solution: The concepts involve the Dynamic Programming in Algorithms.

Given that

X = max weight from the sequence $(a_0, a_1, a_2, \dots$

$$a_{n-1}) = a_0 + \frac{a_1}{2} + \frac{a_2}{4} + \dots + \frac{a_{n-1}}{2^{(n-1)}}$$

Y = max weight from the sequence $(a_1, a_2, \dots, a_{n-1})$

$$= a_1 + \frac{a_2}{2} + \frac{a_3}{4} + \dots + \frac{a_{n-1}}{2^{(n-2)}}$$

Here we have to find X in terms of Y. So,

$$X = \left[a_1 + \frac{a_2}{2} + \frac{a_3}{4} + \dots + \frac{a_{n-1}}{2^{(n-2)}} \right] = Y$$

$$\text{If } \left[a_0 + \frac{a_1}{2} + \frac{a_2}{4} + \dots + \frac{a_{n-1}}{2^{(n-1)}} \right]$$

$$< \left[a_1 + \frac{a_2}{2} + \frac{a_3}{4} + \dots + \frac{a_{n-1}}{2^{(n-2)}} \right]$$

OR

$$X = \left[a_0 + \frac{a_1}{2} + \frac{a_2}{4} + \dots + \frac{a_{n-1}}{2^{(n-1)}} \right] = a_0 + \frac{Y}{2}$$

$$\text{If } \left[a_0 + \frac{a_1}{2} + \frac{a_2}{4} + \dots + \frac{a_{n-1}}{2^{(n-1)}} \right]$$

$$> \left[a_1 + \frac{a_2}{2} + \frac{a_3}{4} + \dots + \frac{a_{n-1}}{2^{(n-2)}} \right]$$

Hence, we sum up as

$$X = \text{MAX}(Y, a_0 + Y/2)$$

Ans. (b)

58. A 5-stage pipelined processor has instruction fetch (IF), instruction decode (ID), operand fetch (OF), perform operation (PO) and write operand (WO) stages. The IF, ID, OF and WO stages take 1 clock cycle each for any instruction. The PO stage takes 1 clock cycle for ADD and SUB instructions, 3 clock cycles for MUL instruction and 6 clock cycles for DIV instruction, respectively. Operand forwarding is used in the pipeline. What is the number of clock cycles needed to execute the following sequence of instructions?

Instruction	Meaning of Instruction
I_0 : MUL R_2, R_0, R_1	$R_2 \leftarrow R_0 \times R_1$
I_1 : DIV R_5, R_3, R_4	$R_5 \leftarrow R_3 / R_4$
I_2 : ADD R_2, R_5, R_2	$R_2 \leftarrow R_5 + R_2$
I_3 : SUB R_5, R_2, R_6	$R_5 \leftarrow R_2 - R_6$

- (a) 13 (b) 15 (c) 17 (d) 19

(GATE 2010: 2 Marks)

Solution:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
I_0	IF	ID	OF	PO	PO	PO	WO								
I_1		IF	ID	OF	—	—	PO	PO	PO	PO	PO	PO	WO		
I_2			IF	ID			OF	—	—	—	—	—	PO	WO	
I_3				IF			ID						OF	PO	WO

With operand forwarding, 15 clock cycles are needed.

Ans. (b)

59. The program below uses six temporary variables a , b , c , d , e , f .

```

a = 1
b = 10
c = 20
d = a + b
e = c + d
f = c + e
b = c + e
e = b + f
d = 5 + e
return d + f

```

Assuming that all operations take their operands from registers, what is the minimum number of registers needed to execute this program without spilling?

- (a) 2 (b) 3
(c) 4 (d) 6

(GATE 2010: 2 Marks)

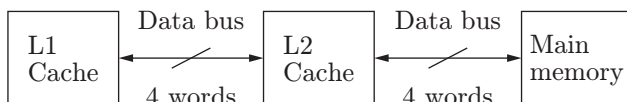
Solution: Let us take three Registers R1, R2 and R3

R1	R2	R3
a = 1	b = 10	c = 20
d = 11	b = 10	c = 20
d = 11	e = 21	c = 20
f = 41	e = 21	c = 20
b = 41	e = 21	c = 20
e = 42	b = 41	f = b = 41
d = 47	e = 42	f = 41

All the operations will be completed using three registers only.

Ans. (b)

Common Data Questions 60 and 61: A computer system has an L1 cache, an L2 cache, and a main memory unit connected as shown below. The block size in L1 cache is 4 words. The block size in L2 cache is 16 words. The memory access times are 2 nanoseconds, 20 nanoseconds and 200 nanoseconds for L1 cache, L2 cache and main memory unit, respectively.



60. When there is a miss in L1 cache and a hit in L2 cache, a block is transferred from L2 cache to L1 cache. What is the time taken for this transfer?

- (a) 2 nanoseconds (b) 20 nanoseconds
(c) 22 nanoseconds (d) 88 nanoseconds

(GATE 2010: 2 Marks)

Solution: Access to L1 cache = 2 ns

Access to L2 cache = 20 ns

Block size of L2 is 16 words; data bus size is 4 words.

So, time taken for this transfer = $4 \times 22 = 88$ ns.

Ans. (d)

61. When there is a miss in both L1 cache and L2 cache, first a block is transferred from main memory to L2 cache, and then a block is transferred from L2 cache to L1 cache. What is the total time taken for these transfers?

- (a) 222 nanoseconds (b) 888 nanoseconds
(c) 902 nanoseconds (d) 968 nanoseconds

(GATE 2010: 2 Marks)

Solution: Block transfer from main memory to L2 cache = $2 + 4 \times (20 + 200) = 2 + 880 = 882$ ns

L2 to L1 = $882 + 20 + 66 = 968$ ns

Ans. (d)

62. A computer handles several interrupt sources of which the following are relevant for this question.

- Interrupt from CPU temperature sensor (raises interrupt if CPU temperature is too high)
- Interrupt from Mouse (raises interrupt if the mouse is moved or a button is pressed)
- Interrupt from Keyboard (raises interrupt when a key is pressed or released)
- Interrupt from Hard Disk (raises interrupt when a disk read is completed)

Which one of these will be handled at the HIGHEST priority?

- (a) Interrupt from Hard Disk
(b) Interrupt from Mouse
(c) Interrupt from Keyboard
(d) Interrupt from CPU temperature sensor

(GATE 2011: 1 Mark)

Solution: Interrupt from CPU temperature sensor will be handled at the highest priority.

Ans. (d)

63. Consider a hypothetical processor with an instruction of type LW $R_1, 20(R_2)$, which during execution reads a 32-bit word from memory and stores it in a 32-bit register R_1 . The effective address of the memory location is obtained by the addition of a constant 20 and the contents of register R_2 . Which of the following best reflects the addressing mode implemented by this instruction for the operand in memory?

- (a) Immediate addressing
(b) Register addressing
(c) Register indirect scaled addressing
(d) Base indexed addressing

(GATE 2011: 1 Mark)

Solution: Effective address = contents of register $R_2 + 20$.

Ans. (d)

64. On a non-pipelined sequential processor, a program segment, which is a part of the interrupt service routine, is given to transfer 500 bytes from an I/O device to memory.

Initialize the address register

Initialize the count to 500

LOOP: Load a byte from device

Store in memory at address given by address register

Increment the address register

Decrement the count

If count != 0 go to LOOP

Assume that each statement in this program is equivalent to a machine instruction which takes one clock cycle to execute if it is a non-load/store instruction. The load-store instructions take two clock cycles to execute.

The designer of the system also has an alternate approach of using the DMA controller to implement the same transfer. The DMA controller requires 20 clock cycles for initialization and other overheads. Each DMA transfer cycle takes two clock cycles to transfer one byte of data from the device to the memory.

What is the approximate speed up when the DMA controller based design is used in place of the interrupt-driven program-based input output?

- (a) 3.4 (b) 4.4
(c) 5.1 (d) 6.7

(GATE 2011: 2 Marks)

Solution: By using load store instructions: $1 + 1 + 500 \times (2 + 2 + 1 + 1 + 1) = 2 + 500 \times 7 = 3502$
By using DMA: $20 + 500 \times 2 = 1020$

Speed up = $3502/1020 = 3.4$

Ans. (a)

65. An 8 KB direct-mapped write-back cache is organized as multiple blocks, each of size 32 bytes. The processor generates 32-bit addresses. The cache controller maintains the tag information for each cache block comprising of the following:

1 Valid bit
1 Modified bit

As many bits as the minimum needed to identify the memory block mapped in the cache.

What is the total size of the memory needed at the cache controller to store metadata (tags) for the cache?

- (a) 4864 bits (b) 6144 bits
(c) 6656 bits (d) 5376 bits

(GATE 2011: 2 Marks)

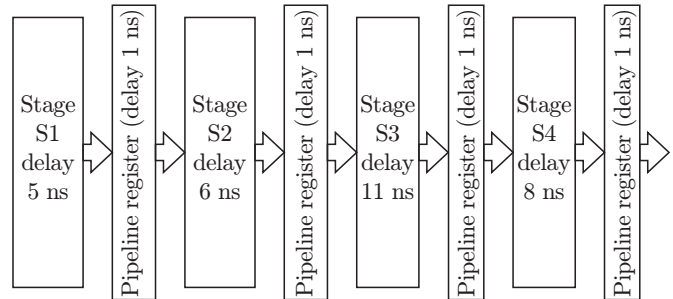
Solution: Number of blocks = $8K/32 = 256$ blocks

Tag bits	Block bits	Word bits
19	8	5

Memory size for tag bits = $19 + 2 = 21$

Total size of memory for tags = $21 \times 256 = 5376$ bits
Ans. (d)

66. Consider an instruction pipeline with four stages (S1, S2, S3 and S4) each with combinational circuit only. The pipeline registers are given in the figure.



What is the approximate speed up of the pipeline in steady state under ideal conditions when compared to the corresponding non-pipeline implementation?

- (a) 4.0 (b) 2.5 (c) 1.1 (d) 3.0

Solution:

$$\text{Speed up} = \frac{\text{Time taken by non-pipelined}}{\text{Time taken by pipelined}} = \frac{5 + 6 + 11 + 8}{11 + 1} = 2.5$$

Ans. (b)

67. The decimal value 0.5 in IEEE single-precision floating-point representation has

- (a) Fraction bits of 000...000 and exponent value of 0
(b) Fraction bits of 000...000 and exponent value of -1
(c) Fraction bits of 100...000 and exponent value of 0
(d) No exact representation

(GATE 2012: 1 Mark)

Solution: Binary representation of 0.5 is 0.1000.

which can be written as $= 1.000 \times 2^{-1}$

Here, exponent = -1 and fraction bits are 000...000.

Ans. (b)

68. Register renaming is done in pipelined processors

- (a) As an alternative to register allocation at compile time
(b) For efficient access to function parameters and local variables
(c) To handle certain kinds of hazards
(d) As part of address translation

(GATE 2012: 1 Mark)

Solution: Register renaming is done to handle WAR/WAW hazards.

Ans. (c)

69. The amount of ROM needed to implement a 4-bit multiplier is

(a) 64 bits (b) 128 bits (c) 1 Kbits (d) 2 Kbits

Solution: Amount of ROM required
 $= 2^{2k} \times 2k$ (where k = number of bits)
 $= 2^{2 \times 4} \times 2 \times 4$
 $= 2 \text{ Kbits}$

Ans. (d)

70. A computer has a 256 KB, 4-way set-associative, write-back data cache with block size of 32 bytes. The processor sends 32-bit addresses to the cache controller. Each cache tag directory entry contains, in addition to address tag, 2 valid bits, 1 modified bit and 1 replacement bit.

The number of bits in the tag field of an address is

(a) 11 (b) 14 (c) 16 (d) 27

(GATE 2012: 2 Marks)

Solution:

$$\text{Number of blocks} = \frac{256 \text{ KB}}{32 \text{ B}} = 2^{13} \text{ blocks}$$

$$\text{Due to 4-way set associative} = 2^{13}/2^2 = 2^{11}$$



Ans. (c)

71. The size of the cache tag directory is

(a) 160 Kbits (b) 136 bits
 (c) 40 Kbits (d) 32 bits

(GATE 2012: 2 Marks)

Solution: Tag directory contains: Tag bits + 4 additional bits = 20 bits

$$\text{Size of cache tag directory} = 20 \times 2^{13} = 160 \text{ Kbits}$$

Ans. (a)

72. In a k -way set associative cache, the cache is divided into v sets, each of which consists of k lines. The lines of a set are placed in sequence one after another. The lines in set s are sequenced before the lines in set $(s+1)$. The main memory blocks are numbered 0 onwards. The main memory block numbered j must be mapped to any one of the cache lines from

(a) $(j \bmod v) * k$ to $(j \bmod v) * k + (k-1)$
 (b) $(j \bmod v)$ to $(j \bmod v) + (k-1)$
 (c) $(j \bmod k)$ to $(j \bmod k) + (v-1)$
 (d) $(j \bmod k) * v$ to $(j \bmod k) * v + (v-1)$

(GATE 2013: 1 Mark)

Solution: Number of sets = v

Number of main memory blocks = j

Number of lines = k (from 0 to $k-1$)

Position will be $(j \bmod v) * k$ to $(j \bmod v) * k + k-1$

Ans. (a)

73. Consider the following sequence of micro-operations.

MBR $\leftarrow$ PC

MAR $\leftarrow$ X

PC $\leftarrow$ Y

Memory $\leftarrow$ MBR

Which one of the following is a possible operation performed by this sequence?

(GATE 2013: 2 Marks)

(a) Instruction fetch
 (b) Operand fetch
 (c) Conditional branch
 (d) Initiation of interrupt service

Solution: Program counter value is stored in memory by MBR and gets a new address by Y. This indicates initialization of interrupt service routine.

Ans. (d)

74. Consider a hard disk with 16 recording surfaces (0-15) having 16384 cylinders (0-16383) and each cylinder contains 64 sectors (0-63). Data storage capacity in each sector is 512 bytes. Data are organized cylinder-wise and the addressing format is <cylinder no., surface no., sector no.>. A file of size 42797 KB is stored in the disk and the starting disk location of the file is <1200, 9, 40>. What is the cylinder number of the last sector of the file, if it is stored in a contiguous manner?

(GATE 2013: 2 Marks)

(a) 1281 (b) 1282 (c) 1283 (d) 1284

Solution: Number of sectors required to store the

$$\text{file} = \frac{42797 \times 1024}{512} = 85594 \text{ sectors}$$

$$\text{Number of sectors in a cylinder} = 16 \times 64 = 1024$$

$$\text{Total number of cylinders required} = \frac{85594}{1024} = 84$$

Last sector will be stored on 1284th cylinder.

Ans. (d)

75. Consider an instruction pipeline with five stages without any branch prediction: Fetch Instruction (FI), Decode Instruction (DI), Fetch Operand (FO), Execute Instruction (EI) and Write Operand (WO). The stage delays for FI, DI, FO, EI and WO are 5 ns, 7 ns, 10 ns, 8 ns and 6 ns, respectively. There are intermediate storage buffers after each stage and the delay of each buffer is 1 ns. A program consisting of 12 instructions $I_1, I_2, I_3, \dots, I_{12}$ is executed in this pipelined processor. Instruction I_4 is the only branch instruction and its branch target is I_9 . If the branch is taken during the execution of this program, the time (in ns) needed to complete the program is

(a) 132 (b) 165 (c) 176 (d) 328

(GATE 2013: 2 Marks)

Solution: Clock pulse duration
 = Maximum stage delay + Buffer delay
 = $11 \times 5 + (4 - 1) \times 11 = 88 \text{ ns}$
 $n + k - 1 = 88 + 88 - 11 = 165$
 Ans. (b)

Common Data Questions 76 and 77: The following code segment is executed on a processor which allows only register operands in its instructions. Each instruction can have at most two source operands and one destination operand. Assume that all variables are dead after this code segment.

```

c = a + b;
d = c * a;
e = c + a;
x = c * c;
if (x > a) {
    y = a * a;
}
else {
    d = d * d;
    e = e * e;
}

```

76. Suppose the instruction set architecture of the processor has only two registers. The only allowed compiler optimization is code motion, which moves statements from one place to another while preserving correctness. What is the minimum number of spills to memory in the compiled code?

- (a) 0 (b) 1
 (c) 2 (d) 3

(GATE 2013: 2 Marks)

Solution: Suppose R_1 and R_2 hold a and b, respectively. Here for code optimization, code motion is allowed. The machine code is;

```

R2 ← R1 + R2    (c = a + b)
R2 ← R2 * R2    (x = c * c)

```

```

if (R2 > R1)
{
    R2 ← R1 * R1    [y = a*a]
}
else
{
    R2 ← c          [read value c from memory]
    R2 ← R2 * R1    [d = c * a;]
    R2 ← R2 * R2    [d = d * d;]
    R2 ← R2 * R1    [e = c + a;]
    R2 ← R2 * R2    [e = e * e;]
}

```

Only 'c' is spilled variable here, which needs to be stored and loaded from memory.

Ans. (b)

77. What is the minimum number of registers needed in the instruction set architecture of the processor to compile this code segment without any spill to memory? Do not apply any optimization other than optimizing register allocation.

- (a) 3 (b) 4 (c) 5 (d) 6

(GATE 2013: 2 Marks)

Solution: Let $R_1 = a$, $R_2 = b$. The machine code is

```

c = a + b;    R2 ← R1 + R2
d = c * a;    R3 ← R2 * R1
e = c + a;    R4 ← R2 + R1
x = c * c;    R2 ← R2 * R2
if (x > a) {   R2 > R1
    y = a * a;  R1 ← R1 * R1
}
else {
    d = d * d;  R3 ← R3 * R3
    e = e * e;  R4 ← R4 * R4
}

```

Four registers are required.

Ans. (b)

PRACTICE EXERCISES

Set 1

1. The three different kinds of buses supporting the architecture of a computer are

- (a) Address bus, data bus, I/O bus
 (b) Front bus, data bus, address bus
 (c) Control bus, data bus, address bus
 (d) I/O bus, address bus, instruction bus

2. The term 'WORD' of a CPU equals to

- (a) The maximum addressable memory size
 (b) The width of a CPU register (integer or float point)

(c) The width of the address bus

(d) The total number of general purpose CPU registers

3. Which of the following statement is false about CISC ARCHITECTURE?

- (a) CISC machine instructions may include complex addressing modes, which require many clock cycles to carry out.
 (b) CISC control units are typically micro-programmed, allowing the instruction set to be more flexible.

- (c) In the CISC instruction set, all arithmetic/logic instructions must be register based for fast processing.
- (d) CISC architectures may perform better in network centric applications than RISC.
4. The register that holds the address of the location to or from which data are to be transferred is called
- Index register
 - Accumulator
 - Memory address registers
 - Memory data registers
5. Which one of the following is not a type of I/O channel?
- Multiplexer
 - Selector
 - Block multiplexer
 - None of the above
6. The performance of a pipelined processor is degraded if _____
- the pipeline stages have different delays
 - consecutive instructions are to be executed serially
 - the pipeline stages share hardware resources
 - all of the above
7. The minimum time delay between the initiation of two independent memory operations is called
- Access time
 - Cycle time
 - Rotational time
 - Latency time
8. The register which keeps track of the execution of a program and which contains the memory address of the instruction currently being executed is known as _____
- index register
 - memory address register
 - program counter
 - instruction registers
9. For interval arithmetic, the best rounding technique used is _____
- rounding to plus and minus infinity
 - rounding to zero
 - rounding to nearest zero
 - rounding to the next number
10. Hardwired control unit are faster than micro-programmed control unit because
- they do not consist of slower memory elements.
 - they do not have slower elements such as gates, flip flops and registers.
 - they consist of elements based on VVLSI design technology.
 - they contain high-speed digital components.
11. The most relevant addressing mode to write position-independent code is
- direct mode
 - auto mode
 - relative mode
 - indexed mode
12. A CPU uses 24-bit instruction. A program starts at address 300 (in decimal). Which one of the following is a legal program counter content (all values in decimal)?
- 324
 - 512
 - 600
 - 700
13. An attempt to access a location not owned by a program is called
- data fault
 - address fault
 - instruction fault
 - page fault
14. Which of the following statement about relative addressing mode is FALSE?
- It enables reduced program code
 - It allows indexing of array element with same instruction
 - It enables easy relocation of data
 - It enables faster address calculation than absolute addressing
15. Compared to CISC processors, RISC processors contain
- more register and smaller instruction set
 - larger instruction set and less registers
 - less registers and smaller instruction set
 - more registers and larger instruction set
16. Micro-programmed control cannot be implemented in RISC architecture because
- it tends to slow down the processor.
 - it consumes more chip areas and large instruction set.
 - handling a large number of registers is impossible in micro-programmed system.
 - the 1 instruction/cycle timing requirement for RISC is difficult to achieve in micro-programmed based architecture.
17. Relocation of the code is easier in _____ irrespective of the program code
- indirect addressing
 - indexed addressing
 - base register addressing
 - absolute addressing
18. In inverted page table organization, the size of the page table depends on
- the number of processes
 - the size of page
 - the size of main memory
 - the number of frames in the main memory

19. When using the concept of locality of reference, the page reference being made by a process
 - (a) will always be to the page used in the previous page reference
 - (b) is likely to be one of the pages used in the past few page references
 - (c) will always be one of the pages existing in the main memory
 - (d) will always lead to page fault
20. If the new version of processor is not made compatible to programs written for its older version, it could be able to process at a faster speed
 - (a) the statement is true.
 - (b) the statement is false.
 - (c) the speed cannot be predicted.
 - (d) speed has nothing to do with the compatibility.
21. A certain snooping cache can snoop only on address line. Which of the following is true?
 - (a) This would adversely affect the system if the write-through protocol is used.
 - (b) It would run well if the write-through protocol is used.
 - (c) Data snooping is mandatory to be implemented on data line.
 - (d) Data snooping may not be required.
22. When the frequency of the input signal to a CMOS gate is increased, the average power dissipation
 - (a) decreases exponentially
 - (b) increases
 - (c) decreases
 - (d) increases exponentially
23. The disadvantage of hardwired control units with flip flop is
 - (a) design becomes complex
 - (b) it requires more number of flip flops
 - (c) control circuit speed does not match with flip flops
 - (d) flip-flops can handle the data unit not the control unit
24. In a vectored interrupt,
 - (a) the branch address is assigned to a fixed location in memory.
 - (b) the interrupting source supplies the branch information to the processor through an interrupt vector.
 - (c) the branch address is obtained from a register in the processor.
 - (d) the branch address is obtained from program counter.
25. A device employing INTA line for device interrupt puts the CALL instruction on the data bus while
 - (a) $\overline{\text{INTA}}$ is active.
 - (b) HOLD is active.
 - (c) $\overline{\text{READY}}$ is active.
 - (d) READY is active.
26. On receiving an interrupt from an I/O device, the CPU
 - (a) branches off to halt (or wait) for a predetermined time
 - (b) branches off to the interrupt service after completion of the current instruction
 - (c) branches off to the interrupt service routine immediately
 - (d) hands over control of address bus and data bus to the interrupting device
27. Using large block size in a fixed block size file system leads to
 - (a) better disk throughput but poorer disk space utilization
 - (b) better disk throughput and better disk space utilization
 - (c) it does not matter as the total memory size is same
 - (d) poorer disk throughput but better disk space utilization
28. Which of the following statements are true about paging?
 - (a) It divides memory into units of equal size.
 - (b) It permits implementation of virtual memory.
 - (c) It suffers from internal fragmentation.
 - (d) It suffers from external fragmentation.
29. The number of entries in an inverted page table
 - (a) is equal to the number of processes.
 - (b) is equal to the number of page frames in the main memory.
 - (c) is equal to the size of the page frame.
 - (d) is equal to the number of page frames in cache memory.
30. In a virtual memory system, the addresses used by the programmer belongs to
 - (a) memory space
 - (b) physical space
 - (c) address space
 - (d) main memory space
31. Power consumption of processors can be vastly reduced by making use of _____ based transistors to implement the ICs.
 - (a) NMOS only
 - (b) TTL Schottky and PMOS
 - (c) PMOS only
 - (d) NMOS and PMOS

- 32.** Address symbol table is generated by the
 (a) memory management software
 (b) assembler
 (c) table match of associative memory
 (d) generated by CPU
- 33.** How many 128×8 RAM chips are needed to have a total RAM of 2048 bytes?
 (a) 8 (b) 16 (c) 24 (d) 32
- 34.** In 8085 microprocessor, how many I/O devices can be interfaced in I/O mapped I/O technique?
 (a) Either 256 input devices or 256 output devices
 (b) 8 I/O devices
 (c) 256 input devices and 256 output devices
 (d) 512 input-output devices
- 35.** After reset, the CPU starts the execution of instruction from memory address
 (a) 1111_H (b) 8000_H
 (c) 0000_H (d) $FFFF_H$
- 36.** In a microprocessor system, suppose TRAP, HOLD and RESET pin got activated at the same time, while the processor was executing some instructions, the system will
 (a) execute the TRAP instruction
 (b) execute the HOLD instruction
 (c) execute the RESET instruction
 (d) none of these instructions will be executed
- 37.** In 8085 microprocessor, the programmer cannot access which flag directly?
 (a) Sign flag (b) Carry flag
 (c) Auxiliary carry flag (d) Parity flag
- 38.** Which of the following is a pseudo-instruction for 8085?
 (a) SPHL (b) CMP
 (c) NOP (d) END
- 39.** The term “cycle stealing” refers to:
 (a) Interrupt-based data transfer
 (b) DMA-based data transfer
 (c) Polling mode data transfer
 (d) Clock cycle overriding
- 40.** Which of the following architecture is not suitable for the following SIMD architecture?
 (a) Vector processor (b) PLA-based processor
 (c) Von Neumann (d) PAL-based processor
- 41.** In a multiprogramming system, which of the following concepts is used?
 (a) Data parallelism (b) Paging
 (c) L1 cache (d) DMA
- 42.** PAL circuit can be defined as
 (a) fixed OR and programmable AND logic.
 (b) programmable OR and programmable AND logic.
 (c) fixed AND and programmable OR logic.
 (d) fixed OR and fixed AND logic.
- 43.** If the clock input applied to a cascaded Mod-6 and Mod-4 counter is 48 kHz. Then the output of the cascaded arrangement shall be of:
 (a) 4.8 kHz (b) 12 kHz
 (c) 8 kHz (d) 48 kHz
- 44.** If there are four ROM ICs of 8K and two RAM ICs of 4K words, then the address range of 1st RAM is (assume initial addresses correspond to ROMs)
 (a) $(8000)_H$ to $(9FFF)_H$ (b) $(5000)_H$ to $(7FFF)_H$
 (c) $(8000)_H$ to $(8FFF)_H$ (d) $(5000)_H$ to $(9FFF)_H$
- 45.** The method for updating the main memory as soon as a word is removed from the cache is called
 (a) Write-through (b) Write-back
 (c) Write-save (d) Cache-save

Set 2

- 1.** The most appropriate matching for the following pairs
- | | |
|------------------------------|--------------|
| X. Indirect addressing | 1. Loops |
| Y. Immediate addressing | 2. Pointers |
| Z. Auto-decrement addressing | 3. Constants |
- (a) X – 3 Y – 2 Z – 1 (b) X – 1 Y – 3 Z – 2
 (c) X – 2 Y – 3 Z – 1 (d) X – 3 Y – 1 Z – 2
- 2.** Which of the following is not a form of memory?
 (a) Instruction cache
 (b) Instruction register
 (c) Instruction opcode
 (d) Translation look aside buffer
- 3.** In serial data transmission, every byte of data is padded with a ‘0’ in the beginning and one or two 1’s at the end of byte because
 (a) Receiver is to be synchronized for byte reception.
 (b) Receiver recovers lost ‘0’ and ‘1’ from these padded bits.
 (c) Padded bits are useful in parity computation.
 (d) None of these.

4. In 2's complement addition, overflow
- is flagged whenever there is carry from sign bit addition
 - cannot occur when a positive value is added to a negative value
 - is flagged when the carries from sign bit and previous bit match
 - none of the above
5. The performance of a pipelined processor suffers if
- the pipeline stages have different delays
 - consecutive instructions are dependent on each other
 - the pipeline stages share hardware resources
 - all of the above
6. Match the pairs in the following questions
By writing the corresponding letters only
- | | |
|--------------|--|
| (A) IEEE 488 | (P) Specifies the interface for connecting a single device |
| (B) IEEE 796 | (Q) Specified the bus standard for connecting a computer to other devices including CPUs |
| (C) IEEE 696 | (R) Specifies the standard for an instrumentation bus |
| (D) RS232-C | (S) Specifies the bus standard for the "back plane" bus called multibus |
- (A) – (P); (B) – (R); (C) – (S); (D) – (Q)
 - (A) – (P); (B) – (Q); (C) – (R); (D) – (S)
 - (A) – (Q); (B) – (S); (C) – (R); (D) – (P)
 - (A) – (R); (B) – (S); (C) – (P); (D) – (R)
7. When an interrupt occurs, an operating system
- Ignores the interrupt
 - Always change state of interrupted process after processing the interrupt
 - Always resumes execution of interrupted process after processing the interrupt
 - May change state of interrupted process to 'blocked' and schedule another process.
8. RAID configuration of disks are used to provide
- Fault-tolerance
 - High speed
 - High data density
 - None of the above
9. Arrange the following configuration for CPU in decreasing order of operating speeds: hardwired control, vertical microprogramming, horizontal microprogramming.
- Hardwired control, vertical microprogramming, horizontal microprogramming.
 - Hardwired control, horizontal microprogramming, vertical microprogramming.

- Horizontal microprogramming, vertical microprogramming, hardwired control.
- Vertical microprogramming, horizontal microprogramming, hardwired control.

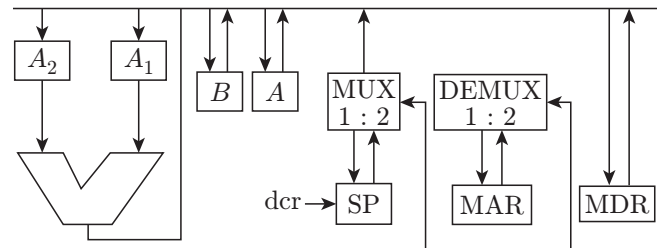
10. A processor needs software interrupt to
- test the interrupt system of the processor
 - implement co-routines
 - obtain system services which need execution of privileged instructions
 - return from subroutine

11. Which is the most appropriate match for the items in the first list with the items in the second list?

List-I	List-II
X. Indirect addressing	I. Array implementation
Y. Index addressing	II. Writing relocatable code
Z. Base register addressing	III. Passing array as parameter

- (X, III) (Y, I) (Z, II)
- (X, II) (Y, III) (Z, I)
- (X, III) (Y, II) (Z, I)
- (X, I) (Y, III) (Z, II)

12. Consider the following data path of a simple non-pipeline CPU. The registers A, B, A₁, A₂, MDR, the bus and the ALU are 8-bit wide. SP and MAR are 16-bit registers. The MUX is of size 8 × (2:1) and the DEMUX is of size 8 × (1:2). Each memory operation takes 2 CPU clock cycles and uses MAR (memory address register) and MDR (memory data register). SP can be decremented locally



The CPU instruction "push r ", where $r = A$ or B has the specification

$$M[sp] \leftarrow r$$

$$SP \leftarrow SP - 1$$

How many CPU clock cycles are needed to execute the "push r " instruction?

- 2
- 3
- 4
- 5

13. In the absolute addressing mode,
- the operand is inside the instruction.
 - the address of the operand is inside the instruction.

- (c) the register containing the address of the operand is specified inside the instruction.
 (d) the location of the operand is implicit.
14. What are the states of the auxiliary carry (AC) and carry flag (CY) after executing the following 8085 program?
- ```
MVI H, 5DH
MIV L, 6BH
MOV A, H
ADD L
```
- (a) AC = 0, CY = 0      (b) AC = 1, CY = 1  
 (c) AC = 1, CY = 0      (d) AC = 0, CY = 1
15. Horizontal microprogramming
- (a) does not require use of signal decoders  
 (b) results in larger-sized microinstructions than vertical microprogramming  
 (c) uses 1 bit for each control signal  
 (d) all of the above
16. For the daisy chain scheme of connecting I/O devices, which of the following statements is true?
- (a) It gives non-uniform priority to various devices.  
 (b) It gives uniform priority to all devices.  
 (c) It is only useful for connecting slow devices to a processor device.  
 (d) It requires a separate interrupt pin on the processor for each device.
17. A microprogram control unit is required to generate a total of 25 control signals. Assume that during any microinstruction at most two control signals are active. Minimum number of bits required in the control word to generate the required control signal is \_\_\_\_\_.
18. The correct matching for the following pairs is
- |                            |                   |
|----------------------------|-------------------|
| A. DMA I/O                 | 1. High-speed RAM |
| B. Cache                   | 2. Disk           |
| C. Interrupt I/O           | 3. Printer        |
| D. Condition code register | 4. ALU            |
- (a) A – 4, B – 3, C – 1, D – 2  
 (b) A – 2, B – 1, C – 3, D – 4  
 (c) A – 4, B – 3, C – 2, D – 1  
 (d) A – 2, B – 3, C – 4, D – 1
19. I/O redirection
- (a) implies changing the name of a file  
 (b) can be employed to use an existing file as input file for a program  
 (c) implies connecting two programs through a pipe  
 (d) none of the above
20. The main difference(s) between a CISC and a RISC processor is/are that a RISC processor typically
- (a) has fewer instruction  
 (b) has fewer addressing modes and more registers  
 (c) is easier to implement using hardwired control logic  
 (d) all of these
21. How big is a 4-way set-associative cache made up of 16 bits word, 4 words per line and having 1024 sets?
- (a) 32 KB      (b) 64 KB  
 (c) 128 KB      (d) 256 KB
21. Consider the expression to be evaluated as  $X = (M + N \times O)/(P \times Q)$ . How many three address instruction are required to evaluate the above given expression?
- (a) 3      (b) 4      (c) 5      (d) 6
23. In  $X = (M + N \times O)/(P \times Q)$ , how many one-address instructions are required to evaluate it?
- (a) 4      (b) 6      (c) 8      (d) 10
24. A decimal number has 64 digits. The number of bits needed for its equivalent binary representation is
- (a) 200      (b) 213      (c) 246      (d) 277
25. Determine the speed up obtained from pipelining if latencies for each stage in single cycle processor is given as:
- |       |       |       |       |       |
|-------|-------|-------|-------|-------|
| IF    | ID    | ALU   | MEM   | WB    |
| 45 ns | 20 ns | 52 ns | 44 ns | 18 ns |

## ANSWERS TO PRACTICE EXERCISES

### Set 1

- |        |        |        |        |         |
|--------|--------|--------|--------|---------|
| 1. (c) | 3. (c) | 5. (d) | 7. (b) | 9. (a)  |
| 2. (b) | 4. (c) | 6. (d) | 8. (c) | 10. (a) |



- |         |         |         |         |         |
|---------|---------|---------|---------|---------|
| 11. (c) | 18. (d) | 25. (a) | 32. (b) | 39. (b) |
| 12. (c) | 19. (b) | 26. (b) | 33. (b) | 40. (c) |
| 13. (b) | 20. (a) | 27. (a) | 34. (c) | 41. (b) |
| 14. (d) | 21. (a) | 28. (c) | 35. (c) | 42. (a) |
| 15. (a) | 22. (b) | 29. (b) | 36. (d) | 43. (a) |
| 16. (a) | 23. (b) | 30. (c) | 37. (c) | 44. (c) |
| 17. (c) | 24. (a) | 31. (d) | 38. (d) | 45. (b) |

## Set 2

- (c) Indirect addressing mode  $\rightarrow$  Pointer access  
Immediate addressing mode  $\rightarrow$  Constant access  
Auto decrement addressing mode  $\rightarrow$  Loops
- (c) An opcode is the portion of a machine language instruction that specifies the operation to be performed.
- (a) Bits in the beginning and at the end of byte are known as start and stop bits, respectively. Start and stop bits are used for synchronization purpose.
- (b) When two 2's complement numbers are added, overflow occurs in the following situations:  
  
Both operands are positive and the result is negative.  
Both operands are negative and the result is positive.  
So option (b) is true, overflow does not occur when positive and negative numbers are added.
- (d) Different hazards are caused due to various dependencies. Different dependencies for pipelined processor are as follows:  
  
Structural dependency is due to different delays in pipelined stages.  
Control dependency is due to consecutive instructions are dependent on each other.  
Data dependency is due to hardware resources sharing.
- (c) (A) IEEE 488 – (Q)  
(B) IEEE 796 – (S)  
(C) IEEE 696 – (R)  
(D) RS232-C – (P)
- (c) When interrupt is caused, the execution of current instruction is stopped. After handling interrupt, the program resumes its execution.
- (a) RAID is random array of independent disks that combines multiple disk drive components into a logical unit. RAID configuration provides fault-tolerance and high speed.
- (b) Horizontal micro programming has high parallelism than vertical. So, the speed order is:

Hardwired control > horizontal microprogramming > vertical microprogramming

- (c) A processor needs software interrupts to obtain system services which need execution of privileged instructions.
- Pointers are used for accessing indirect addresses.  
  
Indirect addressing mode  $\rightarrow$  Pointer access  
In case of immediate addressing, the constant represent the effective address.  
Immediate addressing mode  $\rightarrow$  Constant access  
Auto decrement/increments are used for loop variables.  
Auto decrement addressing mode  $\rightarrow$  Loops
- (a) Push 'r' memory operation needs 2 clocks.
- (b) In absolute addressing mode, the address of the operand is inside the instruction.
- (c)

|         | Carry |   |   |   | Auxiliary carry |   |    |              |
|---------|-------|---|---|---|-----------------|---|----|--------------|
|         | ①     | 1 | 1 | 1 | ①               | 1 | 1  |              |
| (5D) #  | (0    | 1 | 0 | 1 | 1               | 0 | 1) | <sub>B</sub> |
| +(6D) # | +(0   | 1 | 1 | 0 | 1               | 0 | 1) | <sub>B</sub> |
|         | (1    | 1 | 0 | 0 | 1               | 0 | 0) | <sub>B</sub> |

AC = 1 and CY = 0

- (d) Features of horizontal microprogramming are
  - It does not require use of signal decoders
  - It results in larger-sized microinstructions than vertical microprogramming
  - It uses 1 bit for each control signal
- (a) The daisy chaining method of establishing priority consists of a serial connection of all devices that request an interrupt. The device with the highest priority is placed in the first position, followed by lower-priority devices up to the device with the lowest priority, which is placed last in the chain. The farther the device is from the first position, the lower is its priority. Therefore, daisy chain gives non-uniform priority to various devices.

17. (10) To generate 25 control signals, 5 bits are required. To generate two control signals, the following scheme will be used.



5 bits to identify first and 5 bits to identify second including the case when one of them is not present. So total bits required = 10.

18. (b) DMA I/O – Disk  
Cache – High-speed RAM  
Interrupt I/O – Printer  
Condition code register – ALU
19. (c) I/O redirection implies connection two programs through a pipe.
20. (d) The major characteristics of a RISC processor are
- (i) Relatively few instruction
  - (ii) Relatively few addressing modes
  - (iii) More registers
  - (iv) Hardwired rather than microprogrammed control
21. (a) Number of bytes per line =  $16 \text{ bit} \times 4 \text{ byte} = 8 \text{ bytes}$   
Cache size =  $8 \times 4 \times 1024 = 32 \text{ KB}$

22. (b) Three-address instructions are as follow:

MUL R1, N, O  
MUL R2, P, Q  
ADD R3, M, R1  
DIV X, R3, R2

23. (c) LOAD P ( $AC \leftarrow P$ )  
MPY Q ( $AC \leftarrow AC \times Q$ )  
STORE X ( $X \leftarrow AC$ )  
LOAD N ( $AC \leftarrow N$ )  
MPY O ( $AC \leftarrow AC \times O$ )  
ADD M ( $AC \leftarrow AC + M$ )  
DIV X ( $AC \leftarrow AC/X$ )  
STORE X ( $X \leftarrow AC$ )

24. (b) The number of bits is

$$10^{64} - 1 = 2^x - 1 \\ \Rightarrow 10^{64} = 2^x = x = \log_2 10^{64} \approx 213$$

25. (3.44) Speed up obtained by pipelining  
$$= \frac{(45 + 20 + 52 + 44 + 18)}{52} = 3.44$$